

Ethnicity analysis

Ning Liu

2026-04-28

Table of contents

1	Load data	2
2	check umap with ehtnicity labels	11
3	check umap with uncorrected data	15
4	assign a confidence score to each sample	17
4.1	local label agreement & final score	17
4.2	outputing relabels.	22
5	output h5ad	37
6	Benchmarking prediction methods	40
6.1	Seurat label transfer	40
7	Final prediction results	78
8	Markers	78

1 Load data

```
> library(tidyverse)
> library(tidybulk)
> library(SummarizedExperiment)
> library(SingleCellExperiment)
> library(Seurat)
> source("../utils/utils.R")
> library(caret)
> library(ggpubr)
> library(yardstick)
> library(Seurat)
> library(nnet)
> library(ggalluvial)
> library(ggsci)
> library(patchwork)
> library(edgeR)
> library(DelayedMatrixStats)
> library(cellNexus)
> library(igraph)
> library(bluster)
```

```
> se <- HDF5Array::loadHDF5SummarizedExperiment("../data/data_10Mar")
>
> rd <- as.data.frame(rowData(se)) %>%
+   rownames_to_column("gene")
>
```

```

> ethnicities <- c("Europeans", "EastAsian", "SouthAsian", "African", "Hispanic")
>
> top_markers_all <- map(ethnicities, function(eth) {
+
+   est_col <- paste0("Estimate_", eth)
+   se_col <- paste0("Est.Error_", eth)
+   low_col <- paste0("CI.Lower_", eth)
+   up_col <- paste0("CI.Upper_", eth)
+   star_col <- paste0("Star_", eth)
+
+   rd %>%
+     mutate(ethnicity = eth, estimate = .data[[est_col]], std_error =
+       ci_lower = .data[[low_col]], ci_upper = .data[[up_col]], star
+       colnames(.))
+       .data[[star_col]] else NA, dist0_CI = pmax(ci_lower, 0))
+     filter(estimate > 0) %>%
+     arrange(desc(dist0_CI), desc(estimate)) %>%
+     slice_head(n = 200) %>%
+     dplyr::select(ethnicity, gene, estimate, std_error, ci_lower, ci_upper,
+       star)
+ }) %>%
+   bind_rows()
>
> selected_marker_genes <- unique(top_markers_all$gene)
>
> sce_all <- SingleCellExperiment(assay = list(counts = assay(se, 1), gene_
+   2), counts_scaled = assay(se, 3), counts_adjusted_ethnicity = assay(se,
+   colData = colData(se), rowData = rowData(se))
>
> rowData(sce_all)$is_ethnicity_marker <- rownames(sce_all) %in% selected_marker_genes
>
> sce <- sce_all[selected_marker_genes, ]

```

```

> metadata <- get_metadata()
>
> check_eth <- metadata |>
+   dplyr::select(dataset_id, self_reported_ethnicity) %>%
+   as_tibble()
>
> check_eth2 <- metadata |>
+   dplyr::select(dataset_id, sample_id, self_reported_ethnicity, self_reported_ethnicity2) %>%
+   as_tibble()
>
> chosen_samples <- check_eth2 %>%
+   filter(sample_id %in% sce$sample_id) %>%
+   dplyr::select(c("sample_id", "self_reported_ethnicity")) %>%
+   unique()
>
> taxonomy_lookup <- tribble(
+   ~self_reported_ethnicity, ~L1,      ~L2,      ~L3,
+
+   "unknown",                "World",  "Unknown",  "Unknown",
+
+   "African",                "World",  "African",  "Unspecified",
+   "African American",      "World",  "African",  "Diaspora",
+   "African American or Afro-Caribbean",
+   "World",  "African",  "Diaspora",  "Afro-Caribbean"
+
+   "European",              "World",  "European",  "Unspecified",
+   "European American",     "World",  "European",  "Diaspora",
+
+   "Finnish",               "World",  "Nordic",    "Unspecified",
+
+   "British",               "World",  "CelticEnglish", "Unspecified",
+   "Irish",                 "World",  "CelticEnglish", "Unspecified",

```


+	"Jewish Israeli",	"World",	"Jewish",	"Israel",
+				
+	"East Asian",	"World",	"EastAsian",	"Unspecified",
+	"Chinese",	"World",	"EastAsian",	"Chinese",
+	"Han Chinese",	"World",	"EastAsian",	"Chinese",
+	"Singaporean Chinese",	"World",	"EastAsian",	"Chinese diaspora",
+	"Japanese",	"World",	"EastAsian",	"Japan",
+	"Korean",	"World",	"EastAsian",	"Korea",
+				
+	"South Asian",	"World",	"SouthAsian",	"Unspecified",
+	"Bangladeshi",	"World",	"SouthAsian",	"Bangladesh",
+	"Singaporean Indian",	"World",	"SouthAsian",	"Indian diaspora",
+				
+	"South East Asian",	"World",	"EastAsian",	"Malay/SEAsia",
+	"Singaporean Malay",	"World",	"EastAsian",	"Malay/SEAsia",
+				
+	"Hispanic or Latin American",			
+	"World", "Hispanic",	"Spanish/Latin America",		"Hispanic or Lat
+				
+	"Iraqi",	"World",	"Muslim/MENA",	"Arabian Peninsula/
+	"Greater Middle Eastern	(Middle Eastern or North African or Persian)",		
+	"World", "Muslim/MENA",	"MENA/Persian (broad)",		"Greater Middle
+				
+	"Oceanian",	"World",	"Oceanian",	"Oceania",
+	"Pacific Islander",	"World",	"Oceanian",	"Pacific Islander",
+				
+	"Asian",	"World",	"Asian",	"Unspecified",
+	"American",	"World",	"Americas",	"North America",
+				
+	"European,Asian",	"World",	"Mixed",	"Europe + Asia",

```

+   "Native American,Hispanic or Latin American",
+   "World",  "Mixed",          "Native + Hispanic",          "Native American
+ )
>
> chosen_samples <- chosen_samples %>%
+   left_join(taxonomy_lookup, by = "self_reported_ethnicity") %>%
+   mutate(
+     L1   = coalesce(L1, "World"),
+     L2   = coalesce(L2, "Other/Unmapped"),
+     L3   = coalesce(L3, "Other/Unmapped"),
+     Leaf = coalesce(Leaf, self_reported_ethnicity)
+   )

```

```

> unknown_samples <- chosen_samples %>%
+   filter(!L2 %in% c("African", "EastAsian", "European", "Hispanic", "Sou
+ )
>
> chosen_samples <- chosen_samples %>%
+   filter(L2 %in% c("African", "EastAsian", "European", "Hispanic", "Sou
+ )
>
> colnames(sce) <- sce$sample_id
> colnames(sce_all) <- sce_all$sample_id
>
> sce_query <- sce[, sce$sample_id %in% unknown_samples$sample_id]
> sce_query_all <- sce_all[, sce_all$sample_id %in% unknown_samples$sample_
> sce_query <- sce_query_all
>
> sce <- sce[, sce$sample_id %in% chosen_samples$sample_id]
> sce_all <- sce_all[, sce_all$sample_id %in% chosen_samples$sample_id]
>
> sce <- sce[, chosen_samples$sample_id]
> sce_all <- sce_all[, chosen_samples$sample_id]
>

```

```
> all(colnames(sce) == chosen_samples$sample_id)
```

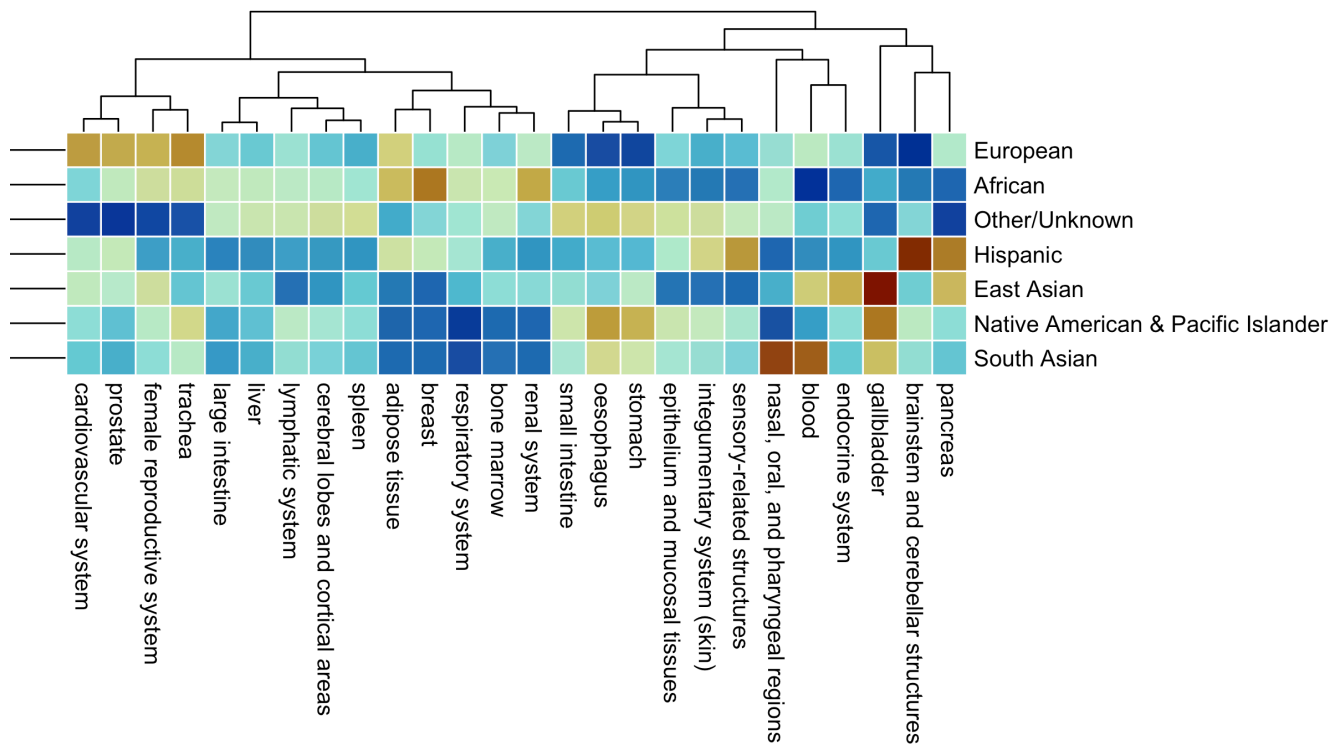
```
[1] TRUE
```

```
> all(colnames(sce_all) == chosen_samples$sample_id)
```

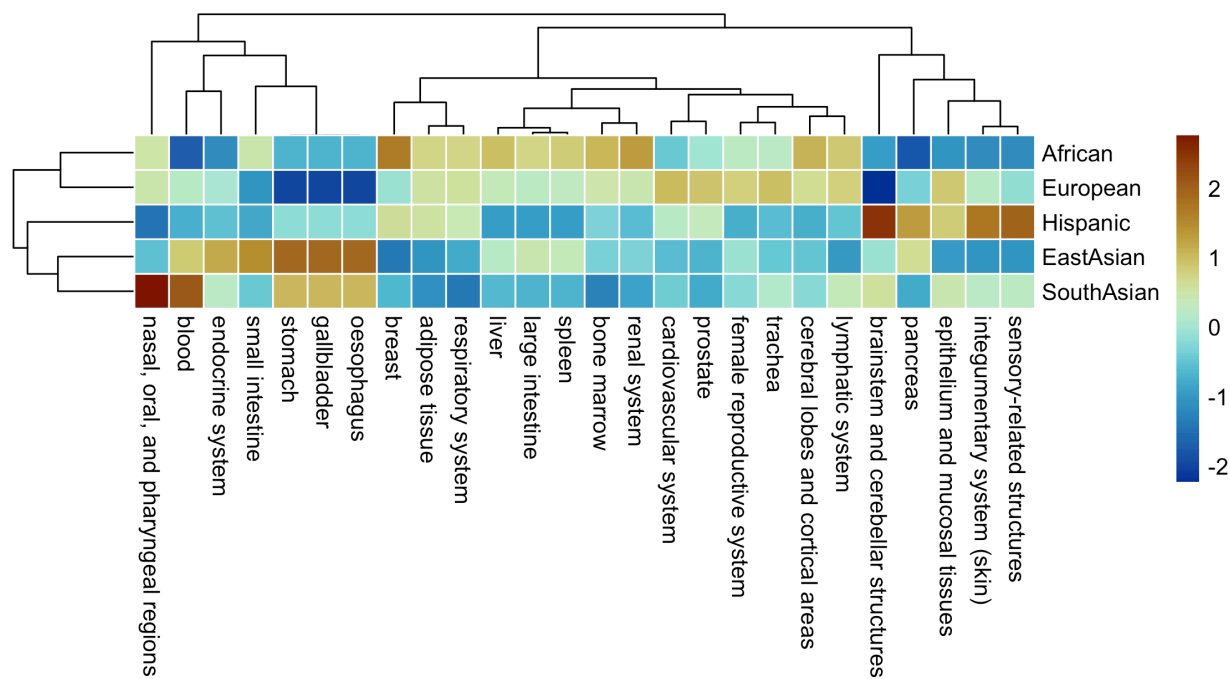
```
[1] TRUE
```

```
> sce$ethnicity_groups <- chosen_samples$L2  
> sce_all$ethnicity_groups <- chosen_samples$L2
```

```
> eth_tissue_heatmap1 <- table(colData(se)[, c("ethnicity_groups", "tissue_  
+   as.matrix() %>%  
+   log1p() %>%  
+   scale() %>%  
+   t() %>%  
+   scale() %>%  
+   t() %>%  
+   pheatmap::pheatmap(border_color = "white", cellwidth = 15, cellheight  
+   direction = -1, palette = "roma"))
```

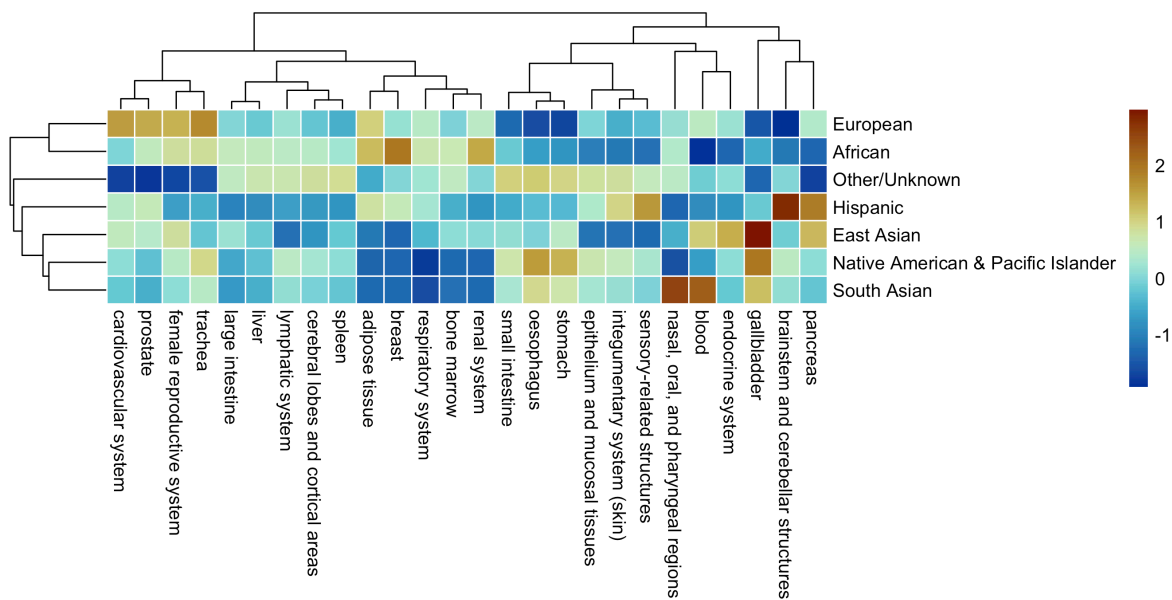


```
> eth_tissue_heatmap2 <- table(colData(sce)[, c("ethnicity_groups", "tissue")],
+   as.matrix() %>%
+   log1p() %>%
+   scale() %>%
+   t() %>%
+   scale() %>%
+   t() %>%
+   pheatmap::pheatmap(border_color = "white", cellwidth = 15, cellheight = 15,
+     direction = -1, palette = "roma"))
```

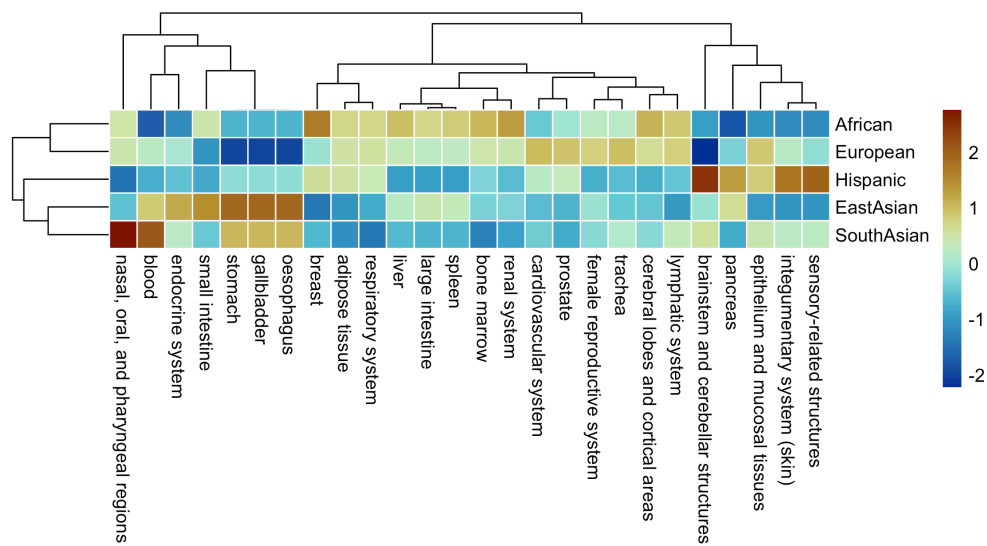


```
> ggplotify::as.ggplot(eth_tissue_heatmap1) + ggplotify::as.ggplot(eth_tiss
+   plot_layout(ncol = 1, widths = c(1, 1)) + plot_annotation(tag_levels =
```

A



B



```
> ggsave("../output/ethnicity_tissue_distribution_heatmap.png", width = 10,
+         bg = "white")
```

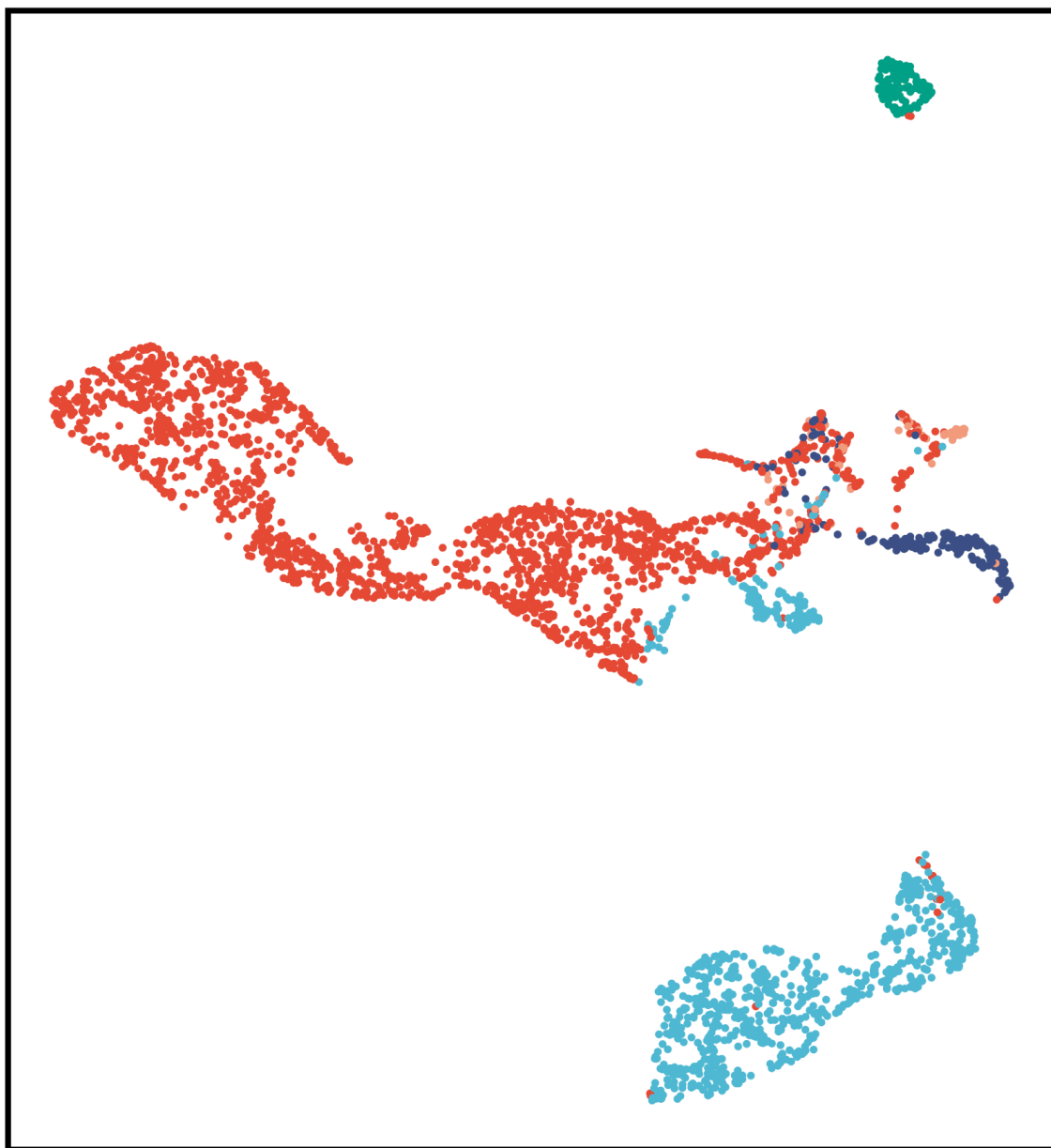
```
> pca <- prcomp(t(assay(sce, "counts_adjusted_ethnicity")), rank. = 30, sca
> umap <- uwot::tmap(pca$x[, 1:10], n_components = 2)
> colnames(umap) <- paste0("UMAP", seq(2))
>
>
> reducedDim(sce, "PCA") <- pca$x
> reducedDim(sce, "UMAP") <- umap
```

```
> npg_cols <- c((ggsci::pal_npg("nrc"))(5), "gray")
> my_colors <- setNames(npg_cols, c("European", "EastAsian", "SouthAsian",
+   "Hispanic", "LowConfidenceLabel"))
```

2 check umap with ethnicity labels

UMAP on adjusted count

```
> standR::plotDR(sce, dimred = "UMAP", color = ethnicity_groups, shape = 16
+   scale_color_manual(values = my_colors) + theme_void() + labs(x = "UMA
+   guides(color = guide_legend(override.aes = list(size = 5), nrow = 3))
+   legend.text = element_text(size = 14), legend.position = "bottom", pa
+   fill = NA, size = 2), plot.margin = unit(c(10, 30, 10, 20), "pt"))
```



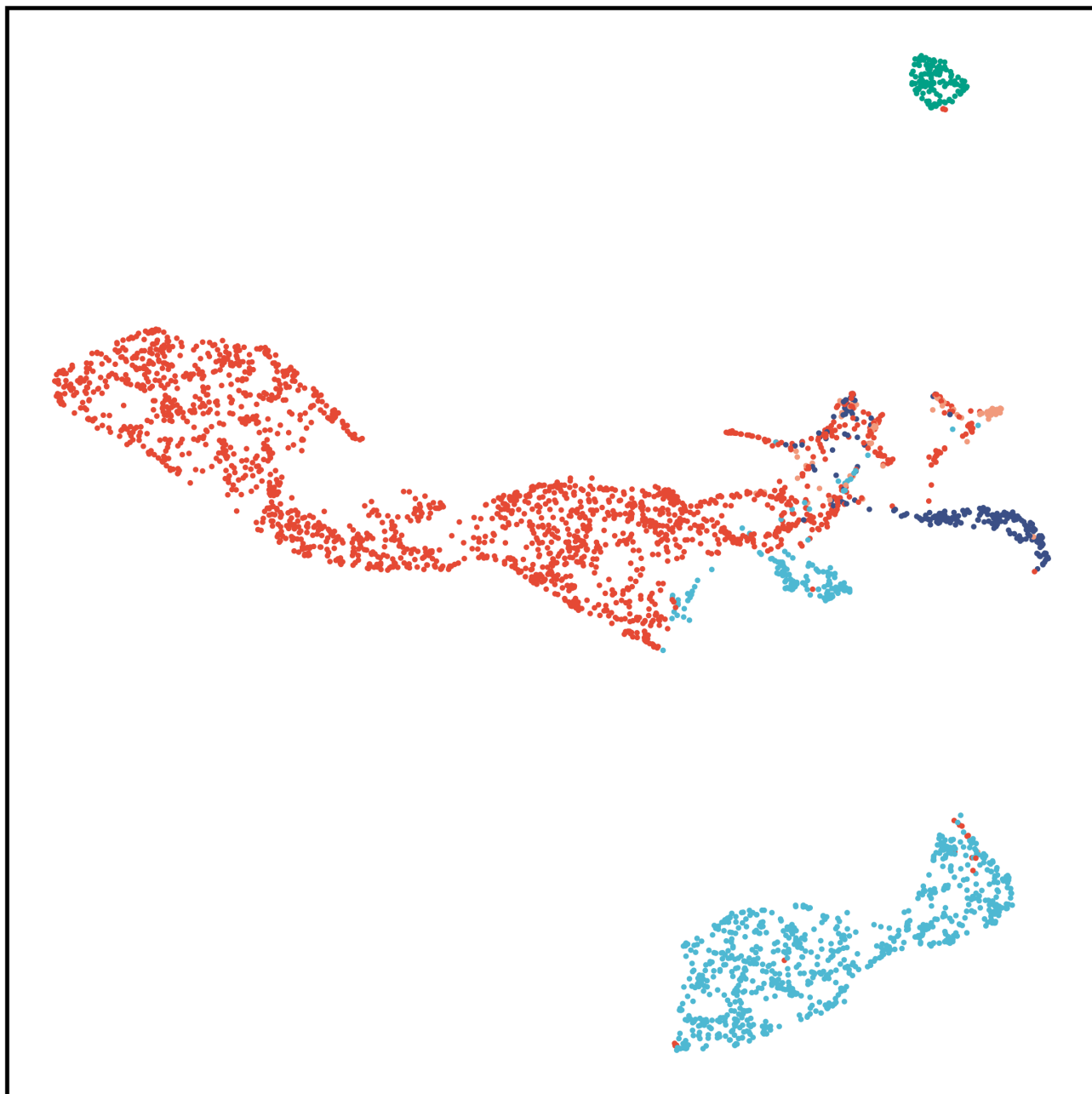
ethnicity_groups

● African	● Hispanic
● EastAsian	● SouthAsian
● European	


```

> p_post_batch_correct <- standR::plotDR(sce, dimred = "UMAP", color = ethn
+   shape = 16, size = 1) + scale_color_manual(values = my_colors) + them
+   labs(x = "UMAP1", y = "UMAP2") + guides(color = guide_legend(override
+   nrow = 3)) + theme(legend.title = element_text(size = 17), legend.tex
+   legend.position = "none", panel.border = element_rect(color = "black"
+   size = 2), plot.margin = unit(c(10, 10, 10, 10), "pt"))
>
≥ p_post_batch_correct

```

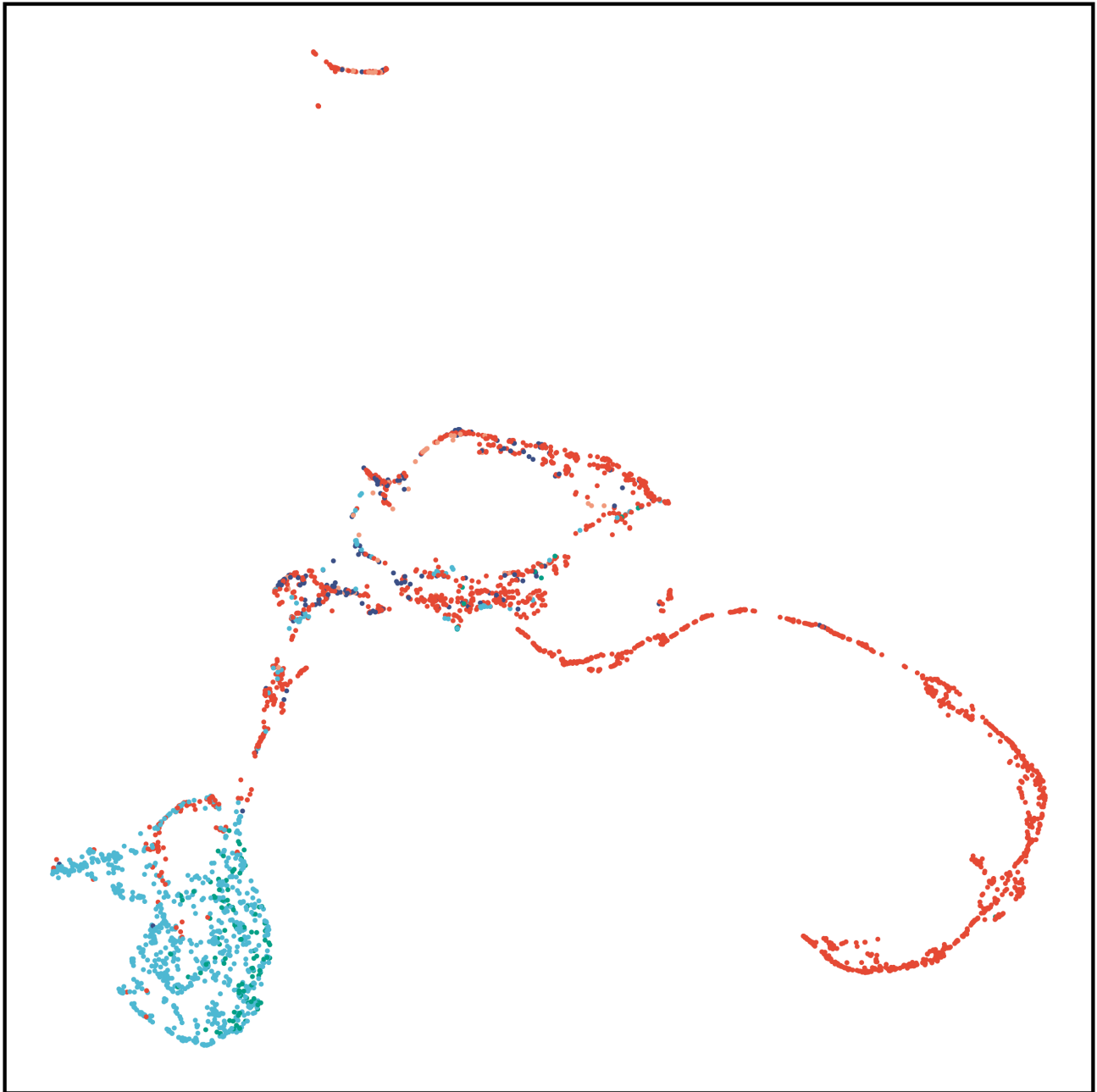


```
> ggsave(p_post_batch_correct, file = "../output/umap_original_adjusted.pdf",
+       height = 7)
```

3 check umap with uncorrected data

```
> sce_ua <- sce
>
> sce_ua <- scater::runPCA(sce_ua, exprs_values = "counts")
> sce_ua <- scater::runUMAP(sce_ua, dimred = "PCA")
```

```
> p_before_batch_correct <- standR::plotDR(sce_ua, dimred = "UMAP", color =
+   shape = 16, size = 1) + scale_color_manual(values = my_colors) + theme
+   labs(x = "UMAP1", y = "UMAP2") + guides(color = guide_legend(override
+   nrow = 4)) + theme(legend.title = element_text(size = 17), legend.tex
+   legend.position = "none", panel.border = element_rect(color = "black"
+   size = 2), plot.margin = unit(c(10, 10, 10, 10), "pt"))
>
> p_before_batch_correct
```



```
> ggsave(p_before_batch_correct, file = "../output/umap_original_unadjusted  
+ height = 7)
```

4 assign a confidence score to each sample

4.1 local label agreement & final score

The previous version penalised every sample in a high-entropy cluster equally. In practice that is too blunt: if a branch point and a clean branch are merged into the same cluster, the whole branch can be pushed into the ambiguous class. To avoid that failure mode, we keep the cluster entropy only as a diagnostic and compute the final confidence score from **cell-level neighbourhood structure**.

For **each sample** i :

1. Find the exact k_{local} nearest neighbours of i in PCA space.
2. Compute the neighbourhood label distribution $p_i(\ell)$.
3. Define:

$$\alpha_i = p_i(L_i)$$

as the local purity for the sample's assigned label, and

$$m_i = \alpha_i - \max_{\ell \neq L_i} p_i(\ell)$$

as the local margin over the strongest competing label.

4. Use the final confidence score

$$\text{score}_i = \alpha_i \times \frac{1 + m_i}{2}.$$

This keeps coherent branches high-confidence when their local neighbourhood is pure, while only the branch junction and other locally mixed regions become ambiguous.

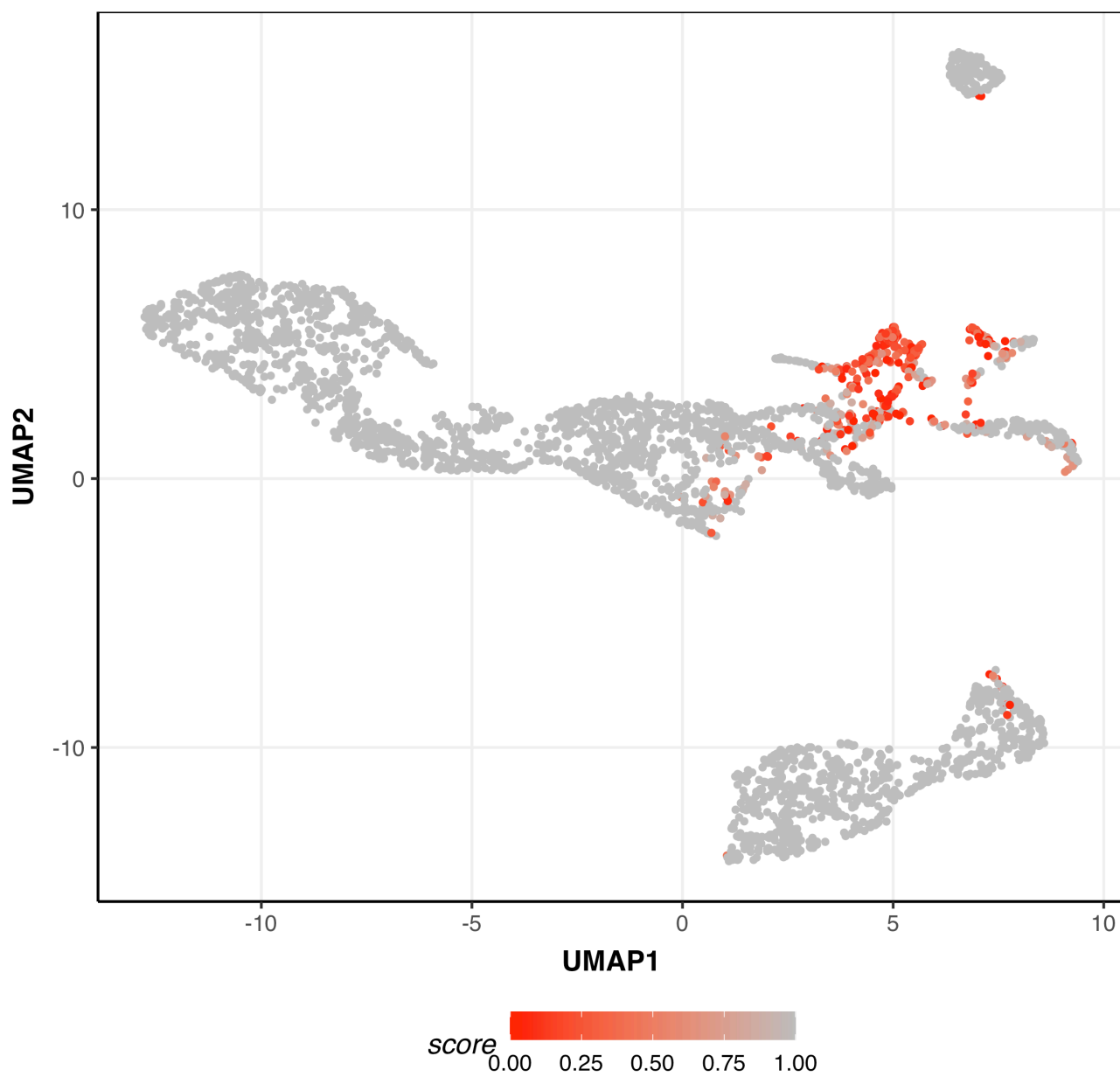
```
> set.seed(10)
>
> pca_data <- pca$x
> df <- colData(sce) %>%
+   as.data.frame()
> k_local <- 15
> knn_res <- BiocNeighbors::findKNN(pca_data, k = k_local)
> nn_index <- knn_res$index
> label_levels <- sort(unique(as.character(df$ethnicity_groups)))
>
> calc_local_label_score <- function(label_i, neighbor_labels, label_levels) {
+   label_i <- as.character(label_i)
+   prob_tbl <- table(factor(neighbor_labels, levels = label_levels))
+   probs <- as.numeric(prob_tbl)/sum(prob_tbl)
+   names(probs) <- label_levels
+
+   alpha_i <- unname(probs[label_i])
+   other_probs <- probs[names(probs) != label_i]
+   runner_up <- if (length(other_probs))
+     max(other_probs) else 0
+   margin_i <- alpha_i - runner_up
+
+   score_i <- alpha_i * ((margin_i + 1)/2)
+
+   tibble(alpha = alpha_i, runner_up = runner_up, margin = margin_i, score = score_i)
+ }
```

```

+ }
>
> local_stats <- vector("list", nrow(pca_data))
>
> for (i in seq_len(nrow(pca_data))) {
+   nn_ids <- nn_index[i, ]
+   neighbor_labels <- as.character(df$ethnicity_groups[nn_ids])
+   local_stats[[i]] <- calc_local_label_score(label_i = df$ethnicity_gro
+     neighbor_labels = neighbor_labels, label_levels = label_levels)
+ }
>
> local_stats <- bind_rows(local_stats)
>
> df <- bind_cols(df, local_stats)
> colData(sce)$alpha <- df$alpha
> colData(sce)$runner_up <- df$runner_up
> colData(sce)$margin <- df$margin
> colData(sce)$local_entropy <- df$local_entropy
> colData(sce)$local_entropy_norm <- df$local_entropy_norm
> colData(sce)$score <- df$score

> standR::plotDR(sce, dimred = "UMAP", color = score, shape = 16, size = 1.
+   high = "gray") + theme_ning() + labs(x = "UMAP1", y = "UMAP2") + them
+   "cm"))

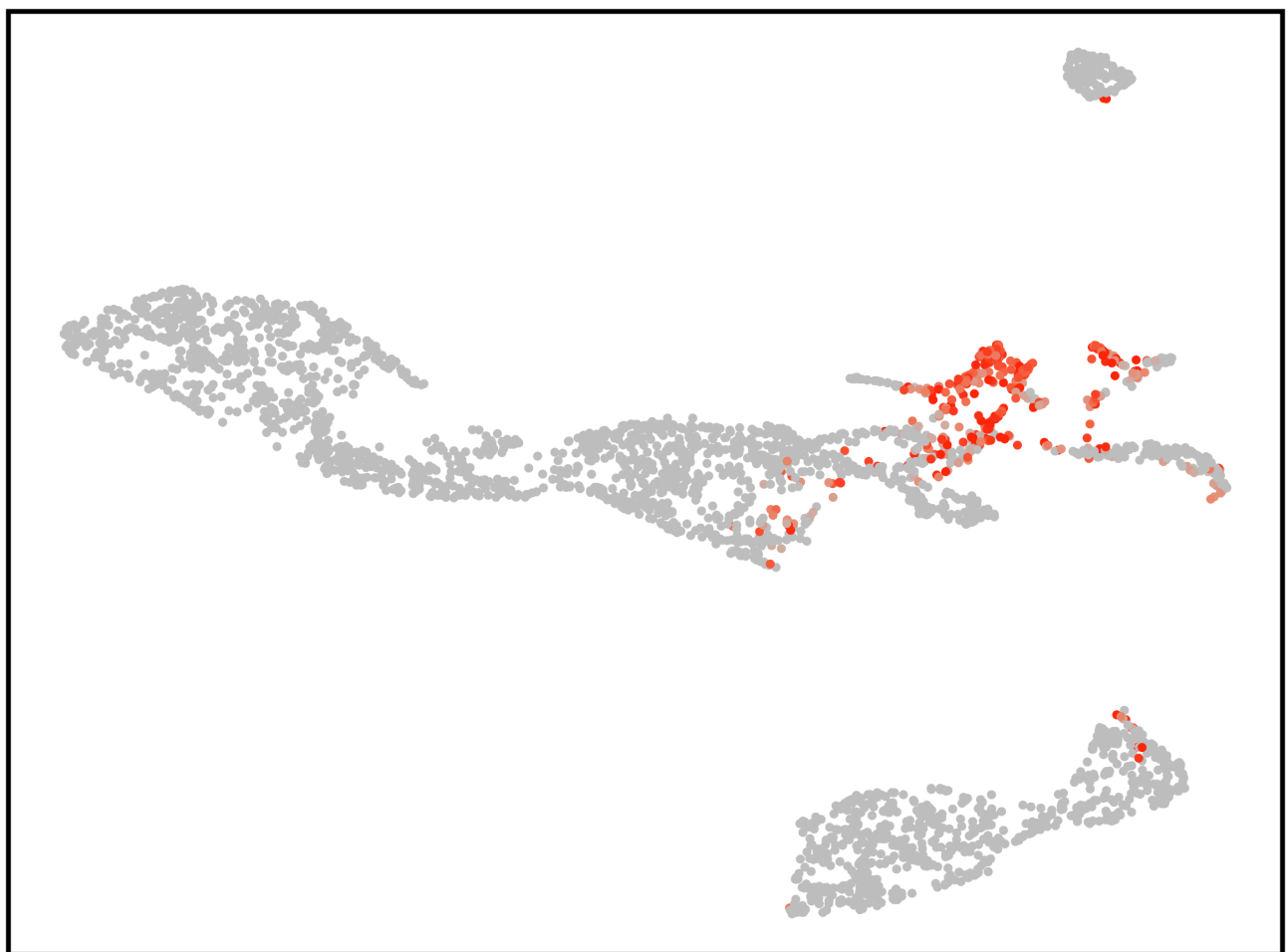
```




```

> p_scoring_umap <- standR::plotDR(sce, dimred = "UMAP", color = score, sha
+   size = 1.5) + scale_color_gradient(low = "red", high = "gray") + them
+   labs(x = "UMAP1", y = "UMAP2") + theme(legend.key.width = unit(1, "cm
+   panel.border = element_rect(color = "black", fill = NA, size = 2), pl
+     10, 10, 10), "pt"))
>
> p_scoring_umap

```

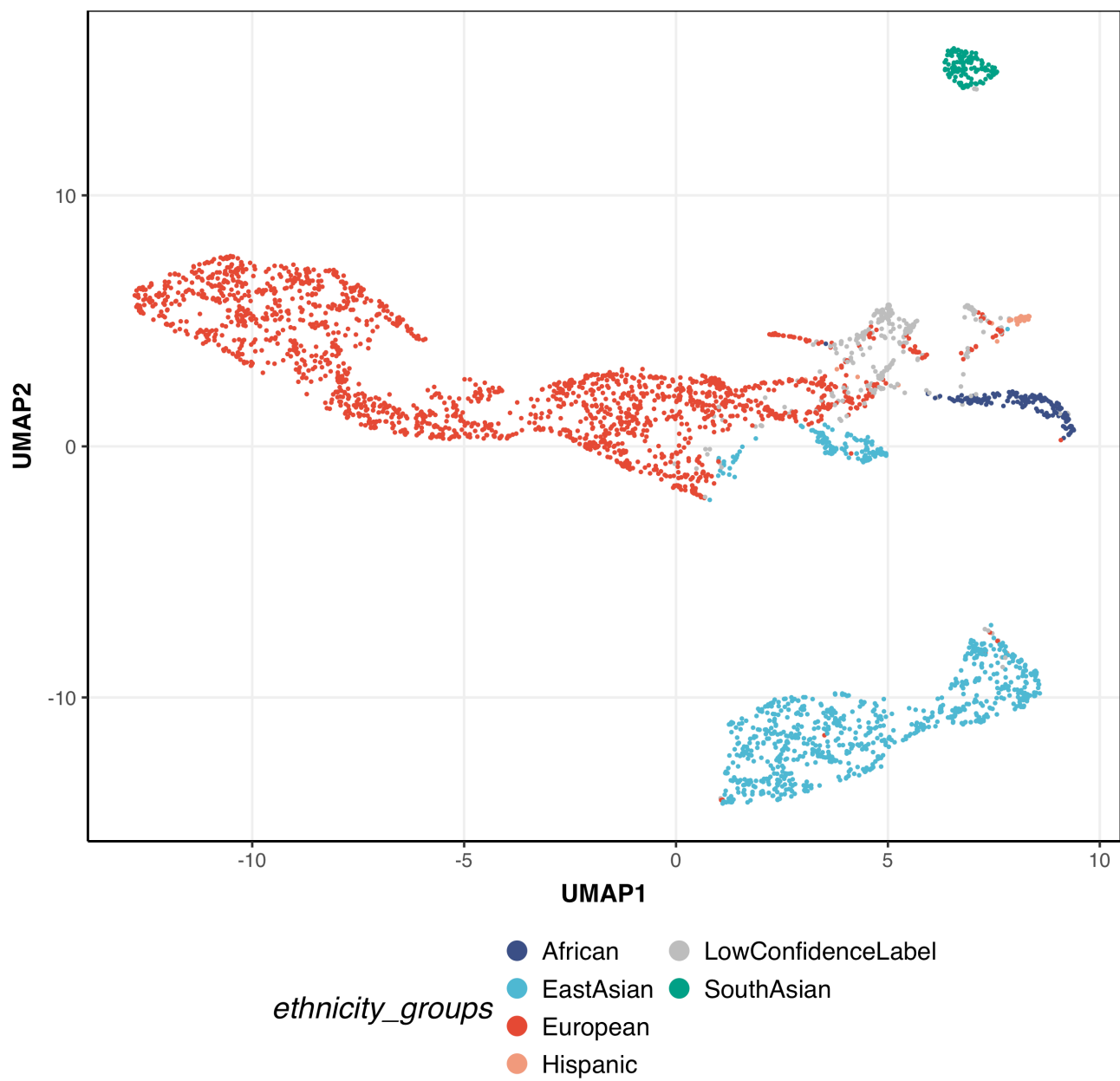


```
> ggsave(p_scoring_umap, file = "../output/umap_score.pdf", width = 8, height = 8)
```

4.2 outputing relabels.

```
> confidence_threshold <- 0.55
>
> sce$ethnicity_relabel <- colData(sce) %>%
+   as.data.frame() %>%
+   mutate(ethnicity_relabel = ifelse(score >= confidence_threshold, as.character(
+     "LowConfidenceLabel"))) %>%
+   .$ethnicity_relabel
>
> colData(sce) %>%
+   as.data.frame() %>%
+   dplyr::select(c("sample_id", "ethnicity_relabel")) %>%
+   mutate(Filter = ifelse(ethnicity_relabel == "LowConfidenceLabel", TRUE, FALSE)) %>%
+   write_csv("../output/sample_filtering_based_on_ethnicity_clearness.csv")
```

```
> standR::plotDR(sce, dimred = "UMAP", color = ethnicity_relabel, shape = 1,
+   scale_color_manual(values = my_colors, name = "ethnicity_groups") + theme(
+   labs(x = "UMAP1", y = "UMAP2") + guides(color = guide_legend(override.aes = TRUE,
+   nrow = 4)) + theme(legend.title = element_text(size = 17), legend.text = element_text(size = 12)))
```



```

> sce$UMAP1 <- reducedDim(sce, "UMAP")[, "UMAP1"]
> sce$UMAP2 <- reducedDim(sce, "UMAP")[, "UMAP2"]
>
>
> colData(sce) %>%
+   as.data.frame() %>%
+   filter(ethnicity_relabel == "EastAsian") %>%
+   filter(UMAP1 < 2) %>%
+   rownames()

```

```

[1] "feebe261621ad9ada100dae715abc6ca"
[2] "e84310ab73d87878a146305751c1567b___AdultAdrenalGland_3.1-1"
[3] "43770e85b1b194215160fce8cdfe2eb4"
[4] "7934579b36067c81e0dad88cbacf6660"
[5] "dc0a765645d542252e2675abfc078add"
[6] "47ea0b9432a16daeb5b59f5ec0932daa"
[7] "846cfad62769748c749db107211299c8"
[8] "635c4ef5104300d292c4abd948154c4d"
[9] "c1c749a3d020711dfe3ca4187173c88e"
[10] "f1d0cf83c223ed8fbcdba1f7e6f77c98"
[11] "aede8072908353e6f25f307cd92fea39"
[12] "656fa95bcc7a207f97bfee9baac3b411"
[13] "d1066f1cbad95fcc1c8dec6a6d5d6da"
[14] "0e3e901f6d0fcfd12874b53bf019413b"
[15] "4462cc165eb3213a954bff284d263be6"
[16] "0f471c8aaa73ec69fece231cc04e696d"
[17] "08c761abf068a4296fb26f03f991ece0"
[18] "8b477609bb94a68e07502f7244cc1aac"
[19] "f31702c90011ef83d5892282ab7316ff"
[20] "e674e33ad0a52e7ddc285bc2b0615762"
[21] "2d8286d4645478f3454ca3d6aad81a3c___AdultSpleenParenchyma_1.-0"
[22] "fe0af29a98f40737d30ab158105ffaaff"

```

[23] "9ce94db1638033402b79448a6ad9b62c"
[24] "355ca95c9a7dc438a3e3a1ee39b531fd"
[25] "3013fec5810777019c234d81a014ad3b"
[26] "c625865677cac6f2f0cffcb5457f3475"
[27] "27dc0760e4334ee8b565e7edda8c90ca"
[28] "a7d362644445c1c68b84310181107bc7"
[29] "742277a65c4c0312f7e3db52aa348909"
[30] "1faabacaa9bb1174bcf29c261bdcf793"
[31] "86a88e42f04cdb3ac69024c6c70a5913"
[32] "d0a10665e4287953ce2409928b9f0fd3"
[33] "543b8990d00c81936f05d572e125655f"
[34] "d475694cb5ec0fbbeb1add06d74712e5d"
[35] "50f9412f47607460a2c0fc31a89c6251"
[36] "d25d7e0f37ce932790962fef5f98d58f"
[37] "0e777684baefa83969f00b49c0c00d5d"
[38] "4508e0197eaff3582a066e180ddbfeaf"
[39] "1f04262644454c8d94714ee32e713120"
[40] "ad2933d8d54c98db4a442e3769fb3054"
[41] "f2b7801d6abc882da36a0dbaf522e388"
[42] "89bc0021797f1e00d522101bb41e2f0b"
[43] "0b8eba4ecd4f06156d41e7dfdfb7afd6"
[44] "93aacb3f60feb4c9efe82f5a16ce8259"
[45] "98b4c361a91209ab3c9c6508ea358e7a"
[46] "d87be6c365a0e69b8a3c7f2efbb6a77c"
[47] "0fbfe069fec7a463c152f3d6c2900cb2"
[48] "9df1a00565a50a01967495c2175db4f0"
[49] "a987bf721ab15ae7a32202a96e9a5e43"
[50] "af8831b1968ad8b6251788e7ef706d2d"
[51] "de0b1a68e8fce1a7e341097bb807a71a"
[52] "a20d35ac292724cfec4be14ccc5adac"
[53] "b302d82e2806398287c890e79be2cf77"
[54] "af70f7ca5fc757a00503980d5a6eff59"

```

[55] "5c5e5336090a5057e0d2cbb19b146953"
[56] "40781edca4df41bf330021d7801d924c"
[57] "aba505eb56758023e4f0aac3b1b68681"
[58] "5d53c76e5f47dcc10dac7c07a8015a38"
[59] "dbfac1cde777960134dd1780b3f1d33d"
[60] "630c27ee862cd8b3d6ba5e043501729f"
[61] "9cb775e5e109a368b49590dccd958911"
[62] "a6d309cb734e0eb443e1a3c75042276e"
[63] "269b69efacb192e5d51397aa141cd880"
[64] "64061e01fcf8475dbd5afae52afa49"
[65] "3faeaa3b434bab2b1180768d41a65211"
[66] "870fdececd348342cdc64ec9b1f21738"
[67] "76376c81a7fc1427d4fb03ef0a93eade"
[68] "923e37f5f119b6c5e40d8b478900cd86"
[69] "54eaba3211f716047aa110563b178438"
[70] "40e4d2598ee32a95a8c0cef252486956"
[71] "83b64b29e1855f32ea6ebf2c64165d79"
[72] "b2f8fe96ef55478977775a14c657ff66"
[73] "2f858f13fbfa0b2dd3ac16cbc0d14eef"
[74] "046f02775d0857877f0786407678d0ed"
[75] "25523e4b293b93bdf08fdb58de73b9e8"
[76] "1722158cb347d6b9aff39bc4561c5820"
[77] "dfef6c6cb0dc2e9c81a22c1d2f3bf99b"
[78] "3241d4fb9c9767adb3714320df3bbe4d"
[79] "02ccc59c92471e0fbf8b4df5d973c319"
[80] "bf99c63b66546bdbbc99b36e5ebda1126"
[ reached 'max' /getOption("max.print") -- omitted 39 entries ]

```

```

> colData(sce) %>%
+   as.data.frame() %>%
+   filter(ethnicity_relabel == "European") %>%
+   filter(UMAP1 > 7) %>%

```

```
+ rownames()
```

```
[1] "244b4d1ffb05429c1e0fc91343afba1d"  
[2] "76a27a0f3edbc6570bd5fee11a96dee9"  
[3] "8b1565f010eb0ed36a5789a96b80799f"  
[4] "189cedc149cfdd5cfea3ab5677d09528"  
[5] "ae48173bc65aea00d4f7f040ad518991___-1-14"  
[6] "d99d6e9019248e82c75261853b5c8125___-1-15"  
[7] "ae48173bc65aea00d4f7f040ad518991___-1-15"  
[8] "ae48173bc65aea00d4f7f040ad518991___-1-16"  
[9] "8dfb1dd8fc9e2da5aca5e7b4d0efdd26"  
[10] "b3f6d1e12d2cb72cd5d41bf63c6c6978"  
[11] "8dc13d96bebce8fadb7ae4ac1cacbb28"  
[12] "1d94b5ddb79c68449d323f3404c276dd"  
[13] "d99d6e9019248e82c75261853b5c8125___-1-17"  
[14] "ae48173bc65aea00d4f7f040ad518991___-1-17"  
[15] "c544ba4fc1984288e4a4ba0e76981778"  
[16] "d99d6e9019248e82c75261853b5c8125___-1-18"  
[17] "d99d6e9019248e82c75261853b5c8125___-1-19"  
[18] "3557ac08d4947a11e2ff56fede541077___KC33_1"
```

```
> samples2check <- c(colData(sce) %>%  
+   as.data.frame() %>%  
+   filter(ethnicity_relabel == "EastAsian") %>%  
+   filter(UMAP1 < 2) %>%  
+   rownames() %>%  
+   head(n = 2), colData(sce) %>%  
+   as.data.frame() %>%  
+   filter(ethnicity_relabel == "European") %>%  
+   filter(UMAP1 > 7) %>%  
+   rownames() %>%  
+   head(n = 2))
```

```

> samples2check_df <- reducedDim(sce, "UMAP")[samples2check, ] %>%
+   cbind(colData(sce) %>%
+     as.data.frame() %>%
+     .[samples2check, ]) %>%
+   as.data.frame() %>%
+   dplyr::select(c("sample_id", "UMAP1", "UMAP2", "ethnicity_relabel", "

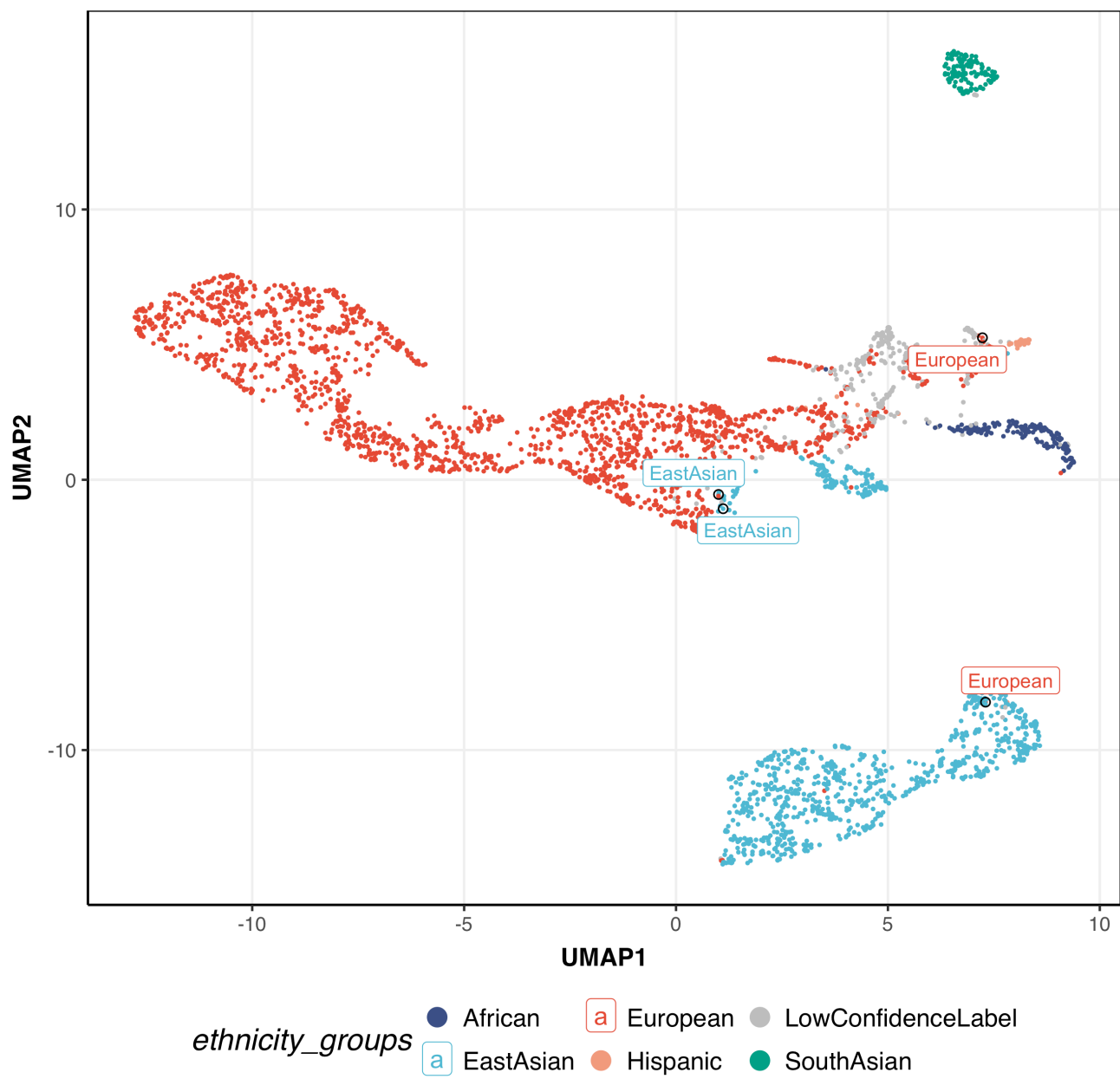
```

6 suspicious samples are selected.

```

> p_sus1 <- standR::plotDR(sce, dimred = "UMAP", color = ethnicity_relabel,
+   size = 0.8) + scale_color_manual(values = my_colors, name = "ethnicity")
+   geom_point(data = samples2check_df, aes(x = UMAP1, y = UMAP2), color = "red",
+     shape = 1, size = 2) + ggrepel::geom_label_repel(data = samples2check_df,
+   aes(x = UMAP1, y = UMAP2, label = ethnicity_relabel)) + theme_ning() +
+   y = "UMAP2") + guides(color = guide_legend(override.aes = list(size = 12)))
+   theme(legend.title = element_text(size = 17), legend.text = element_text(size = 12))
>
> p_sus1

```

```
> ggsave("../output/sus_case_umap.png", width = 11, height = 11, bg = "white")
```

```
> set.seed(10)
> g <- makeKNNGraph(pca_data, k = 10, directed = FALSE)
> V(g)$name <- rownames(pca_data)
>
> dfl <- list()
>
> for (i in samples2check) {
+   v_index <- which(V(g)$name == i)
+   node_neighbors <- neighbors(g, v_index)
+   neighbor_names <- V(g)$name[node_neighbors]
+   df1 <- data.frame(target = i, sampleID = c(i, neighbor_names))
+
+   dfl[[i]] <- df1
+ }
```

```
> plot_list <- lapply(dfl, function(x) {
+   as.data.frame(colData(sce[, x$sampleID])) %>%
+     rownames_to_column("sampleID") %>%
+     left_join(x) %>%
+     ggplot(aes(UMAP1, UMAP2, color = ethnicity_groups)) + geom_point(
+     aes(UMAP1, UMAP2), color = "gray", size = 0.1) + geom_point(size = 0.1) +
+     geom_point(data = samples2check_df %>%
+       filter(sample_id == unique(x$target)), aes(UMAP1, UMAP2), color =
+       shape = 1, size = 2.5) + scale_color_manual(values = my_colors)
+     theme_ning() + labs(x = "UMAP1", y = "UMAP2") + guides(color = guide_legend(
+     nrow = 1)) + theme(legend.title = element_text(size = 12), legend.position =
+     theme(legend.position = "none")
+   })
```

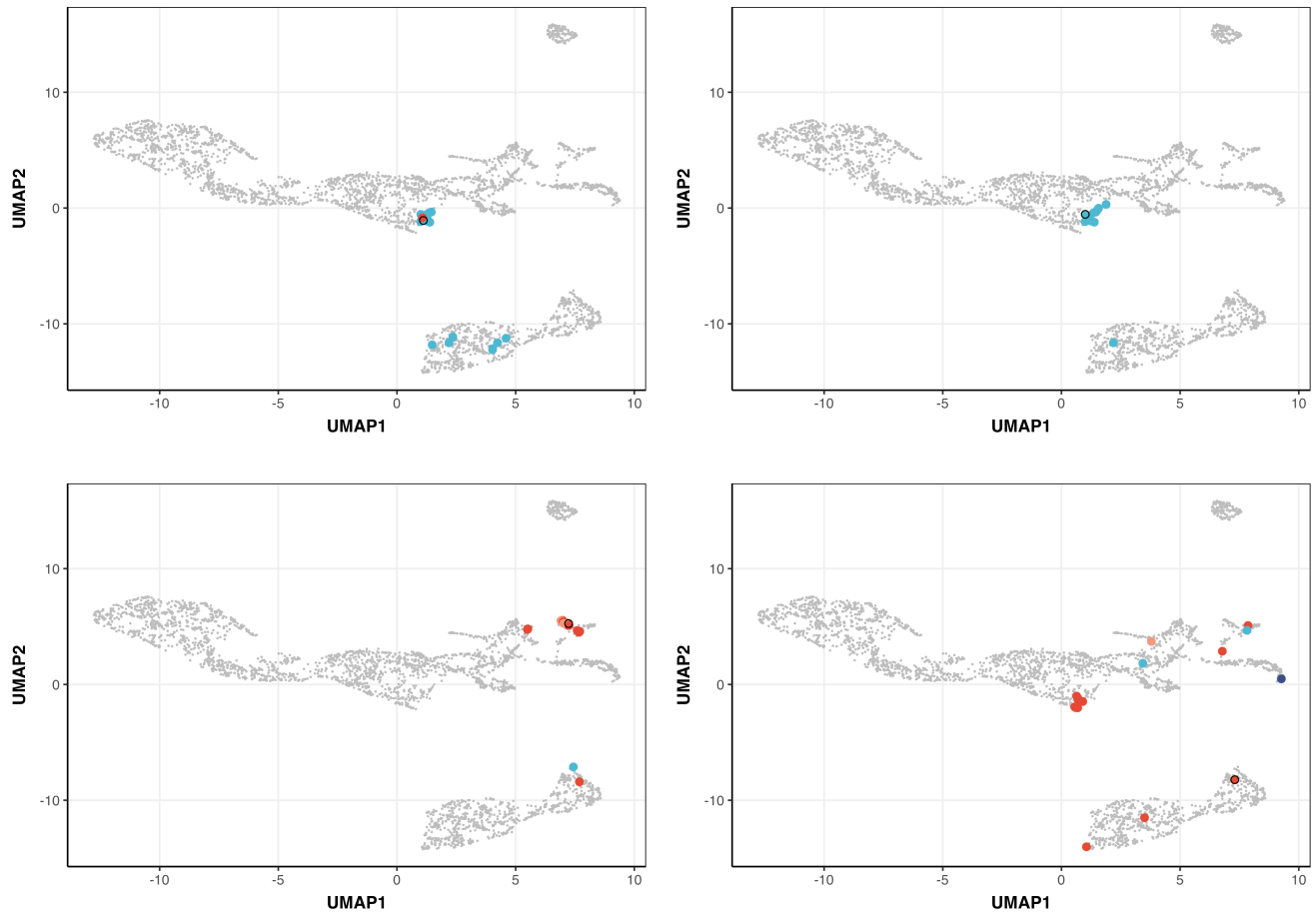
We can see the neighbors of each sample are of the same colors, indicating the

same ethnicity, so they should have high confidence, consistent with scoring algorithm. The misplacement on the UMAP space is just because it's not a linear transformation.

Red circles are the 6 suspicious samples.

```
> p_sus2 <- ggpubr::ggarrange(plotlist = plot_list)
> p_sus2 <- annotate_figure(p_sus2, top = text_grob("Visualise neighbors of
+       color = "black", face = "bold", size = 15))
>
> p_sus2
```

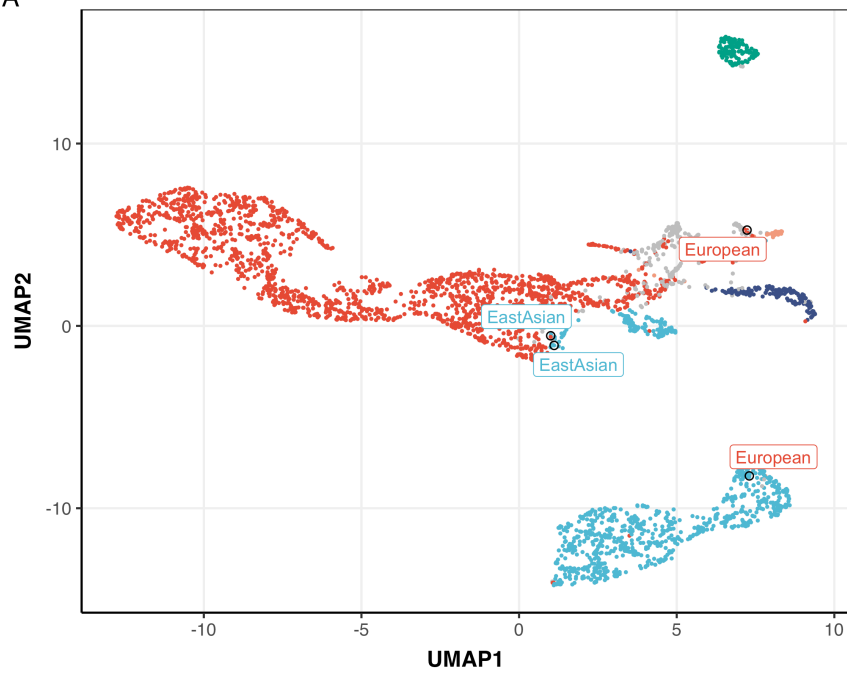
Visualise neighbors of specific samples with the kNN-Graph



```
> ggsave("../output/sus_case_umap_knn_highlight.png", width = 15, height = 10)

> p_sus1 + p_sus2 + plot_layout(ncol = 1, heights = c(1, 1)) + plot_annotat
```

A

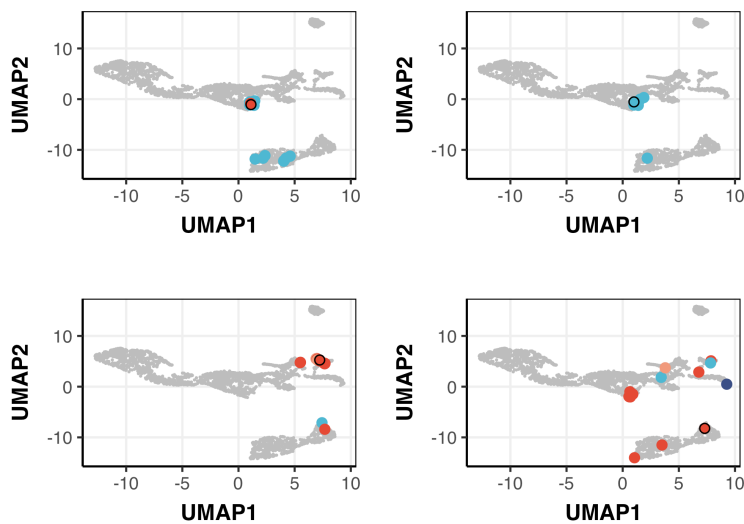


ethnicity_groups

- African
- EastAsian
- European
- Hispanic
- LowConfidenceLabel
- SouthAsian

B

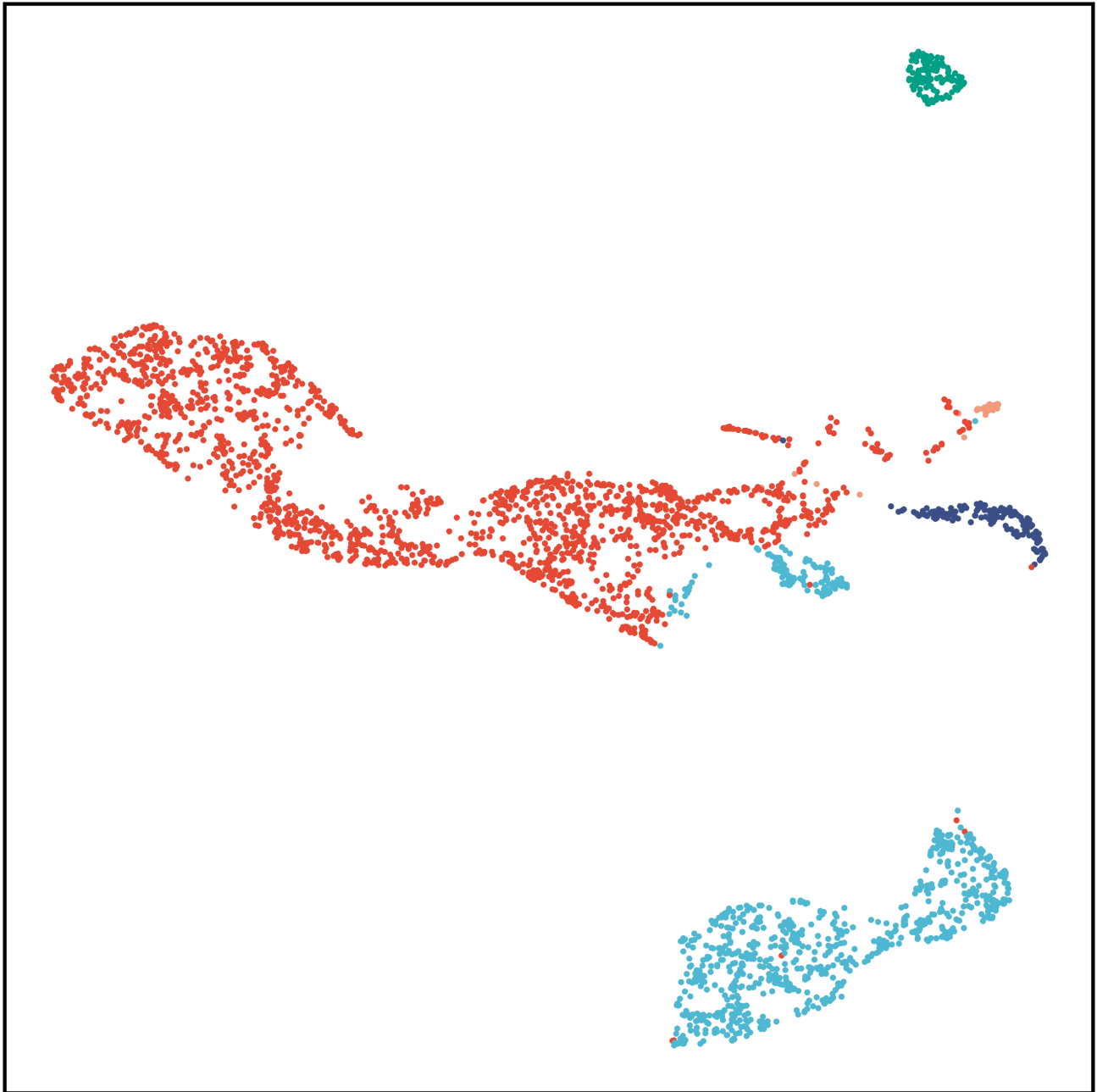
Visualise neighbors of specific samples with the kNN-Graph



```
> ggsave("../output/suspicious_sample_qc_umaps.png", width = 7, height = 13,
```

Here we plot without the low confidence label samples.

```
> p_onlyhighscore <- standR::plotDR(sce[, sce$ethnicity_relabel != "LowConfidence"],  
+   dimred = "UMAP", color = ethnicity_relabel, shape = 16, size = 1.3) +  
+   name = "ethnicity_groups") + theme_ning() + labs(x = "UMAP1", y = "UMAP2") +  
+   guides(color = guide_legend(override.aes = list(size = 8, shape = 15))) +  
+   theme(legend.title = element_text(size = 20), legend.text = element_text(size = 14),  
+     axis.text = element_text(size = 18), axis.title = element_text(size = 14),  
+     theme_void() + theme(legend.position = "none", panel.border = element_rect(fill = NA, size = 2),  
+     plot.margin = unit(c(10, 10, 10, 10), "pt"))  
>  
≥ p_onlyhighscore
```



```

> ggsave(p_onlyhighscore, file = "../output/UMAP_highConfidence.pdf", width
+
+
+
> transfer_cols <- c("ethnicity_groups", "clusters", "entropy", "entropy_st
+   "runner_up", "margin", "local_entropy", "local_entropy_norm", "score"
+
+
> for (col_nm in transfer_cols) {
+   colData(sce_all)[[col_nm]] <- colData(sce)[[col_nm]][match(colnames(s
+     colnames(sce))]
+ }
+
>
> add_training_assays <- function(sce_obj) {
+   raw_counts <- as.matrix(assay(sce_obj, "counts"))
+
+   assay(sce_obj, "logcounts") <- edgeR::cpm(raw_counts, log = TRUE)
+
+   sce_tmm <- sce_obj %>%
+     tidybulk::scale_abundance(method = "TMM", suffix = "_tmm")
+
+   if (!"counts_tmm" %in% assayNames(sce_tmm)) {
+     stop("counts_tmm assay was not created by tidybulk::scale_abundan
+   }
+
+   counts_tmm <- as.matrix(assay(sce_tmm, "counts_tmm"))
+
+   assay(sce_obj, "counts_tmm") <- counts_tmm
+   assay(sce_obj, "logcounts_tmm") <- log1p(counts_tmm)
+   assay(sce_obj, "cpm_tmm_logcounts") <- edgeR::cpm(counts_tmm, log = T
+
+   sce_obj
+ }
+
>
> sce_all <- add_training_assays(sce_all)

```



```
> sce_query_all <- add_training_assays(sce_query_all)
```

5 output h5ad

```
> zellkonverter::writeH5AD(sce_all, file = "../data/sce_relabel.h5ad", comp
>
> sce <- zellkonverter::readH5AD("../data/sce_relabel.h5ad")
```

```
> sce_ref <- sce[, sce$ethnicity_relabel != "LowConfidenceLabel"]
```

```
> sce_l <- sce[, sce$ethnicity_relabel == "LowConfidenceLabel"]
```

```
> sce_query
```

```
class: SingleCellExperiment
dim: 12789 2011
metadata(0):
assays(4): counts gene_presence counts_scaled counts_adjusted_ethnicity
rownames(12789): ENSG000000000003 ENSG000000000457 ... ENSG00000285427
               ENSG00000288859
rowData names(28): is_gene_shared .abundant ...
               Star_HispanicDLatinAmerican is_ethnicity_marker
colnames(2011): 0013454cf3bd3caf513407fab32beca1
               002ee55b6dec45e166be1c940ecbcdcf ... ffb833f3042a1449d574497493823315
               ffcefc95fc366ea78a3d9f8324ba9e9a
colData names(22): sample_id is_immune ... disease_groups__altered
               offset
```

```
reducedDimNames(0):  
mainExpName: NULL  
altExpNames(0):
```

```
> intersect_cols <- intersect(colnames(colData(sce_1)), colnames(colData(sce_query)))
```

```
> cold_merge_query <- colData(sce_1)[, intersect_cols] %>%  
+   rbind(colData(sce_query)[, intersect_cols]) %>%  
+   as.data.frame()
```

```
> sce_query <- SingleCellExperiment(assay = list(counts = cbind(as.data.frame(cold_merge_query[, intersect_cols]),  
+   "counts")), as.data.frame(assay(sce_query, "counts")), counts_adjusted = cbind(cold_merge_query[, intersect_cols],  
+   "counts_adjusted_ethnicity")), as.data.frame(assay(sce_query, "counts")), counts_scaled = cbind(as.data.frame(assay(sce_1, "counts_scaled")), as.data.frame(cold_merge_query[, intersect_cols]),  
+   "counts_scaled")), gene_presence = cbind(as.data.frame(assay(sce_1, "gene_presence")), as.data.frame(assay(sce_query, "gene_presence"))), colData = colData(sce_1))  
>  
> seu_query <- CreateSeuratObject(counts = as.matrix(assay(sce_query, "counts")), meta.data = as.data.frame(assay(sce_query, "counts_adjusted_ethnicity")), assay = "RNA", project = "Seurat")  
>  
> seu_query@assays$RNA$data <- as.matrix(assay(sce_query, "counts_scaled"))  
> seu_query@assays$RNA$var <- as.matrix(assay(sce_query, "counts_scaled"))
```

```
> set.seed(42)  
>  
> pca_q <- prcomp(t(assay(sce_query, "counts_adjusted_ethnicity")), rank. = 10)  
> umap_q <- uwot::tmap(pca_q$x[, 1:10], n_components = 2)  
> colnames(umap_q) <- paste0("UMAP", seq(2))  
>  
> reducedDim(sce_query, "PCA") <- pca_q$x  
> reducedDim(sce_query, "UMAP") <- umap_q
```

```
> p_query_umap <- standR::plotDR(sce_query, dimred = "UMAP", color = "black",  
+   size = 1) + theme_void() + theme(panel.border = element_rect(color =  
+   fill = NA, size = 2), plot.margin = unit(c(10, 10, 10, 10), "pt"))  
>  
> p_query_umap
```



```
> ggsave(p_query_umap, file = "../output/UMAP_query.pdf", width = 8, height = 8)
```

```
> zellkonverter::writeH5AD(sce_query_all, file = "../data/sce_query.h5ad",
>
> sce_query <- zellkonverter::readH5AD(file = "../data/sce_query.h5ad")
```

Now need to run the Classifier_benchmark.ipynb file

6 Benchmarking prediction methods

6.1 Seurat label transfer

```
> sce_ref <- sce[, sce$ethnicity_relabel != "LowConfidenceLabel"]
> assay(sce_ref, "logcounts") <- as.matrix(assay(sce_ref, "logcounts"))
> assay(sce_ref, "counts") <- as.matrix(assay(sce_ref, "counts"))
> seu_ref <- as.Seurat(sce_ref)
> levels_order <- unique(sce_ref$ethnicity_relabel)
> clean_levels <- make.names(levels_order, unique = TRUE)
> clean_to_ori_name <- levels_order
> names(clean_to_ori_name) <- clean_levels
> seu_ref$ethnicity_clean <- factor(seu_ref$ethnicity_relabel, levels = levels_order,
+   labels = clean_levels)
> split_seed <- 52
> n_folds <- 5
> benchmark_recipe_names <- c("LogCPMScale", "LogCPMPCA", "Log1pTMMScale",
+   "LogCPM_TMMScale", "LogCPM_TMMPCA")
> selected_models <- c(as.vector(t(outer(c("HistGBDT", "LinearSVM", "LogReg",
+   "RandomForest"), benchmark_recipe_names, paste, sep = "__"))), "RandomForest")
```

```

> build_group_holdout_folds <- function(meta_df, group_col, k = 5, seed = 5
+   set.seed(seed)
+
+   if (!group_col %in% colnames(meta_df)) {
+     stop(group_col, " is required in meta_df.")
+   }
+
+   df <- meta_df %>%
+     filter(!is.na(.data[[group_col]]), !is.na(ethnicity)) %>%
+     transmute(holdout_group = as.character(.data[[group_col]]), ethn
+
+   all_ethnicities <- sort(unique(df$ethnicity))
+
+   group_stats <- df %>%
+     dplyr::count(holdout_group, ethnicity, name = "n") %>%
+     pivot_wider(names_from = ethnicity, values_from = n, values_fill =
+
+   group_sizes <- df %>%
+     dplyr::count(holdout_group, name = "n_samples")
+
+   group_stats <- group_sizes %>%
+     left_join(group_stats, by = "holdout_group")
+
+   class_group_counts <- colSums(as.matrix(group_stats[, all_ethnicities
+     0])
+
+   coverage_list <- setNames(lapply(seq_len(nrow(group_stats)), function
+     all_ethnicities[as.numeric(group_stats[i, all_ethnicities, drop =
+     0]
+   )), group_stats$holdout_group)
+
+   group_names <- group_stats$holdout_group

```

```

+   protected_lookup <- tolower(protected_groups)
+   candidate_group_names <- group_names[!tolower(group_names) %in% prote
+   total_samples <- sum(group_stats$n_samples)
+   target_test_n <- ceiling(total_samples/k)
+   usage_counts <- setNames(integer(length(group_names)), group_names)
+   folds <- vector("list", k)
+   heldout_groups <- vector("list", k)
+
+   for (i in seq_len(k)) {
+     selected <- character()
+     covered <- character()
+     current_n <- 0
+
+     repeat {
+       remaining_ethnicities <- setdiff(all_ethnicities, covered)
+       candidates <- setdiff(candidate_group_names, selected)
+
+       if (!length(remaining_ethnicities) && current_n >= target_test_n)
+         break
+     }
+     if (!length(candidates)) {
+       break
+     }
+
+     candidate_tbl <- tibble(holdout_group = candidates) %>%
+       mutate(gain = map_int(holdout_group, ~sum(remaining_ethni
+         coverage_list[[.x]])), rarity_gain = map_dbl(holdout_gr
+       gained <- intersect(remaining_ethnicities, coverage_lis
+       if (!length(gained)) {
+         return(0)
+       }
+       sum(1/class_group_counts[gained])

```

```

+       }), keeps_train_cover = map_lgl(holdout_group, function(g
+         train_groups <- setdiff(group_names, c(selected, grp))
+         train_cover <- unique(unlist(coverage_list[train_groups.
+         all(all_ethnicities %in% train_cover)
+       }), usage = usage_counts[holdout_group], n_samples = group
+         group_names)], size_gap = abs((current_n + n_samples) -
+         tie_break = runif(n())) %>%
+       filter(keeps_train_cover)
+
+     if (!nrow(candidate_tbl)) {
+       break
+     }
+
+     if (length(remaining_ethnicities)) {
+       candidate_tbl <- candidate_tbl %>%
+         arrange(desc(gain), desc(rarity_gain), usage, size_gap,
+     } else {
+       candidate_tbl <- candidate_tbl %>%
+         arrange(usage, size_gap, desc(rarity_gain), tie_break)
+     }
+
+     chosen <- candidate_tbl$holdout_group[1]
+     selected <- c(selected, chosen)
+     covered <- union(covered, coverage_list[[chosen]])
+     current_n <- current_n + group_stats$n_samples[match(chosen,
+     usage_counts[chosen] <- usage_counts[chosen] + 1L
+   }
+
+   train_groups <- setdiff(group_names, selected)
+   train_cover <- unique(unlist(coverage_list[train_groups]))
+
+   folds[[i]] <- which(as.character(meta_df[[group_col]]) %in% selec

```

```

+       heldout_groups[[i]] <- selected
+   }
+
+   names(folds) <- paste0("Fold", seq_len(k))
+   attr(folds, "heldout_groups") <- heldout_groups
+   folds
+ }
+
> make_split_plan <- function(seu_obj, group_col, k = 5, seed = 52) {
+   group_values <- as.character(seu_obj@meta.data[[group_col]])
+   fold_meta <- tibble(sample_id = colnames(seu_obj), ethnicity = as.character(
+     group_value = group_values)
+   colnames(fold_meta)[colnames(fold_meta) == "group_value"] <- group_col
+
+   protected_groups <- if (identical(group_col, "tissue_groups"))
+     "blood" else character()
+
+   folds <- build_group_holdout_folds(fold_meta, group_col = group_col,
+     protected_groups = protected_groups)
+
+   for (i in seq_along(folds)) {
+     test_i <- folds[[i]]
+     train_i <- setdiff(seq_len(ncol(seu_obj)), test_i)
+     stopifnot(length(intersect(unique(group_values[test_i]), unique(group_values[train_i]))
+       0)
+     stopifnot(all(clean_levels %in% unique(as.character(seu_obj$ethnicity[train_i]))
+     stopifnot(all(clean_levels %in% unique(as.character(seu_obj$ethnicity[test_i]))
+   }
+
+   list(group_col = group_col, folds = folds, heldout_groups = attr(folds, "heldout_groups"),
+     group_values = group_values)
+ }

```



```

>
> split_plan_list <- list(dataset_id = make_split_plan(seu_ref, group_col =
+   k = n_folds, seed = split_seed), tissue_groups = make_split_plan(seu_
+   k = n_folds, seed = split_seed))
>
> folds <- split_plan_list$dataset_id$folds
> heldout_dataset_list <- split_plan_list$dataset_id$heldout_groups
> groups <- split_plan_list$dataset_id$group_values

> cv_split_group_df <- imap_dfr(split_plan_list, function(plan, split_scheme)
+   map2_dfr(plan$folds, seq_along(plan$folds), ~{
+     test_i <- .x
+     train_i <- setdiff(seq_len(ncol(seu_ref)), test_i)
+
+     bind_rows(tibble(split_scheme = split_scheme, fold = .y, split =
+       holdout_group = plan$group_values[train_i]), tibble(split_scheme =
+       fold = .y, split = "Test", holdout_group = plan$group_values[
+     })
+ }) %>%
+   dplyr::count(split_scheme, fold, split, holdout_group, name = "n_samples")
+   group_by(split_scheme, fold, split) %>%
+   mutate(prop_samples = n_samples/sum(n_samples)) %>%
+   ungroup()
>
> cv_split_group_df

```

A tibble: 290 x 6

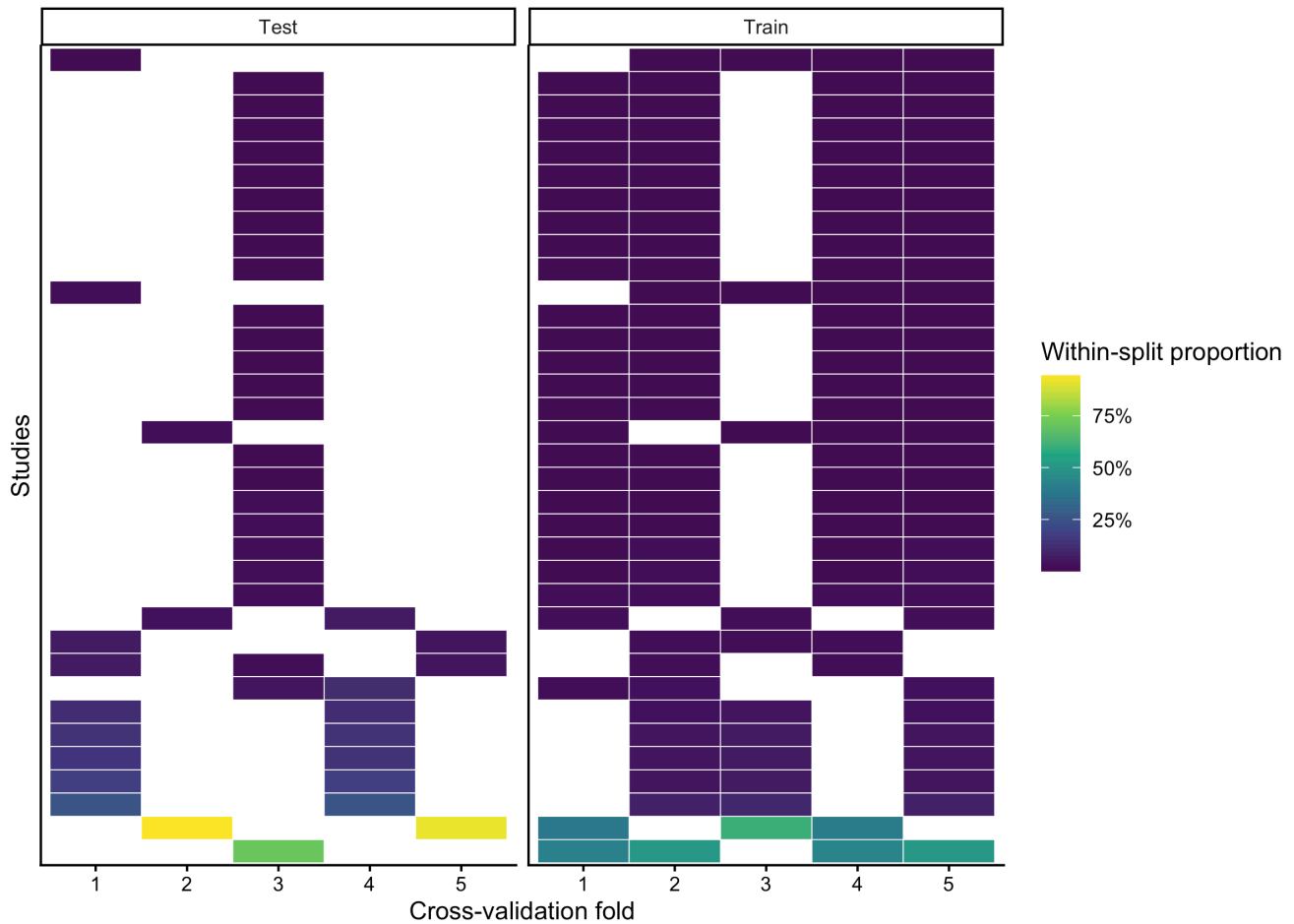
	split_scheme	fold	split	holdout_group	n_samples	prop_samples
	<chr>	<int>	<chr>	<chr>	<int>	<dbl>
1	dataset_id	1	Test	01ad3cd7-3929-4654-84c0-6db0~	40	0.0345
2	dataset_id	1	Test	093d3bfe-6f0f-4ac0-a7a1-829f~	106	0.0919

3	dataset_id	1	Test	1e5bd3b8-6a0e-4959-8d69-cafe~	1	0.
4	dataset_id	1	Test	2a498ace-872a-4935-984b-1afa~	41	0.
5	dataset_id	1	Test	3c361813-5d5e-4868-bfe5-e759~	12	0.
6	dataset_id	1	Test	842c6f5d-4a94-4eef-8510-8c79~	153	0.
7	dataset_id	1	Test	9f222629-9e39-47d0-b83f-e08d~	89	0.
8	dataset_id	1	Test	d4e69e01-3ba2-4d6b-a15d-e704~	87	0.
9	dataset_id	1	Test	dea717d4-7bc0-4e46-950f-fd7e~	71	0.
10	dataset_id	1	Train	0b4a15a7-4e9e-4555-9733-2423~	18	0.

i 280 more rows

```
> holdout_group_order <- cv_split_group_df %>%
+   group_by(split_scheme, holdout_group) %>%
+   summarise(n_total = sum(n_samples), .groups = "drop") %>%
+   arrange(split_scheme, desc(n_total))
>
> cv_split_group_plot_df <- cv_split_group_df %>%
+   left_join(holdout_group_order, by = c("split_scheme", "holdout_group"))
+   group_by(split_scheme) %>%
+   mutate(holdout_group = factor(holdout_group, levels = rev(unique(holdout_group))))
+   ungroup()
>
> ggplot(cv_split_group_plot_df %>%
+   filter(split_scheme == "dataset_id"), aes(x = factor(fold), y = holdout_group,
+   fill = prop_samples)) + geom_tile(color = "white") + facet_wrap(~split_scheme,
+   nrow = 1) + scale_fill_viridis_c(labels = scales::percent_format(accuracy = 1))
+   theme_classic() + theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())
+   labs(x = "Cross-validation fold", y = "Studies", fill = "Within-split proportion")
+   title = "Train/test group composition across held-out folds")
```

Train/test group composition across held-out folds

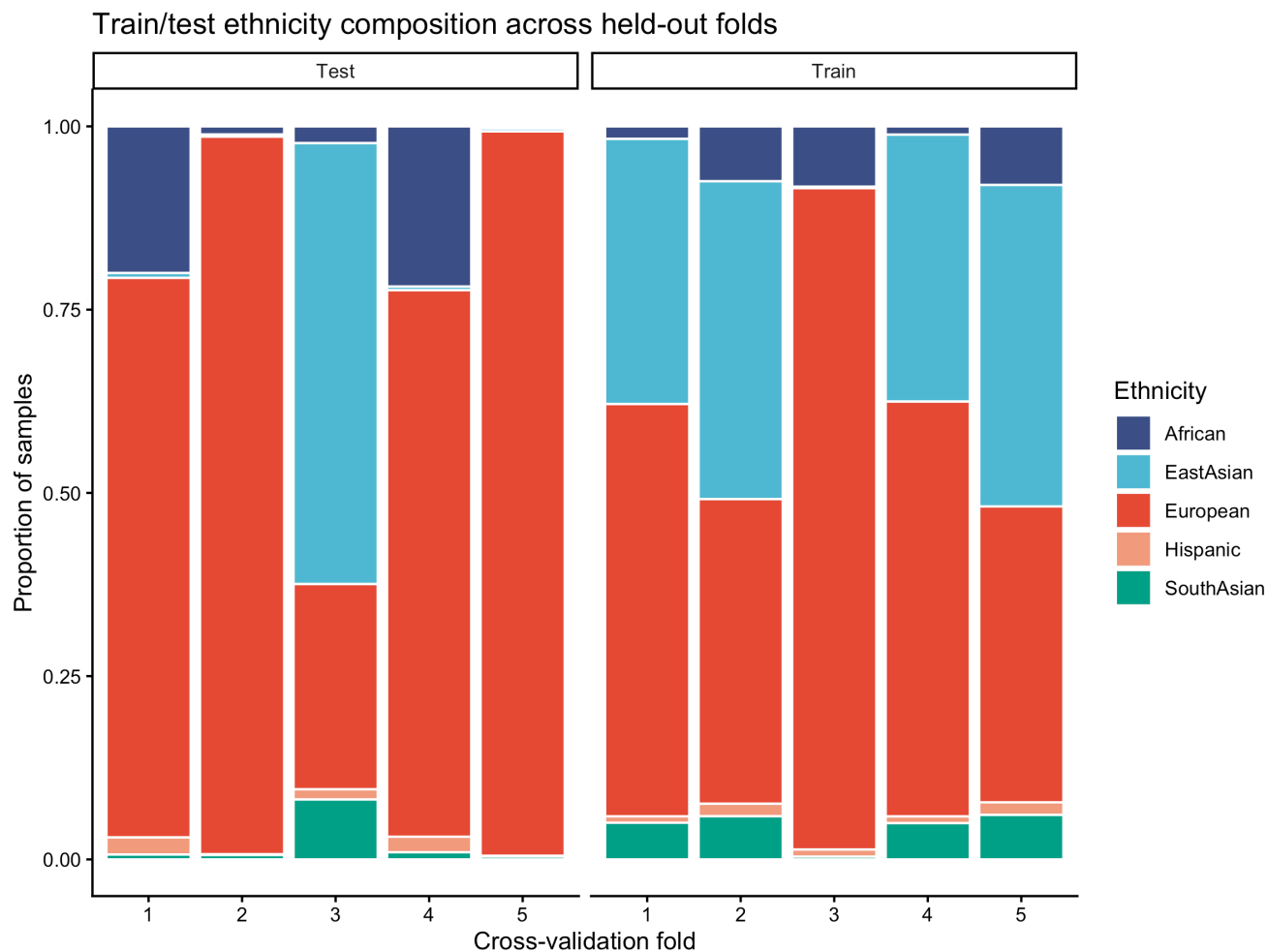


```
> ggsave("../output/study_held_out_distribution_check.pdf", width = 7, height = 10,
+         bg = "white")
>
> cv_split_ethnicity_df <- imap_dfr(split_plan_list, function(plan, split_scheme) {
+   map2_dfr(plan$folds, seq_along(plan$folds), ~{
+     test_i <- .x
+     train_i <- setdiff(seq_len(ncol(seu_ref)), test_i)
+
+     bind_rows(tibble(split_scheme = split_scheme, fold = .y, split = "train") %>%
+       filter(train_i == fold),
+               tibble(split_scheme = split_scheme, fold = .y, split = "test") %>%
+                 filter(test_i == fold))
+   })
+ })
```

```

+         ethnicity = as.character(seu_ref$ethnicity_relabel[train_i]))
+         fold = .y, split = "Test", ethnicity = as.character(seu_ref$e
+     })
+ }) %>%
+   dplyr::count(split_scheme, fold, split, ethnicity, name = "n_samples"
+   group_by(split_scheme, fold, split) %>%
+   mutate(prop_samples = n_samples/sum(n_samples)) %>%
+   ungroup()
+
>
≥ ggplot(cv_split_ethnicity_df %>%
+   filter(split_scheme == "dataset_id"), aes(x = factor(fold), y = prop_
+   fill = ethnicity)) + geom_col(color = "white") + facet_wrap(~split) +
+   unique(cv_split_ethnicity_df$ethnicity])) + theme_classic() + labs(x =
+   y = "Proportion of samples", fill = "Ethnicity", title = "Train/test

```



```
> ggsave("../output/study_held_out_ethnicity_distribution_check.pdf", width
+         bg = "white")
```

```
> sample_expression_values <- function(sce_obj, assay_name, sample_set, dis
+   split_scheme, fold, n_values = 25000, transform_fn = identity) {
+   vals <- as.numeric(assay(sce_obj, assay_name))
+   vals <- vals[is.finite(vals)]
+
+   if (length(vals) > n_values) {
```

```

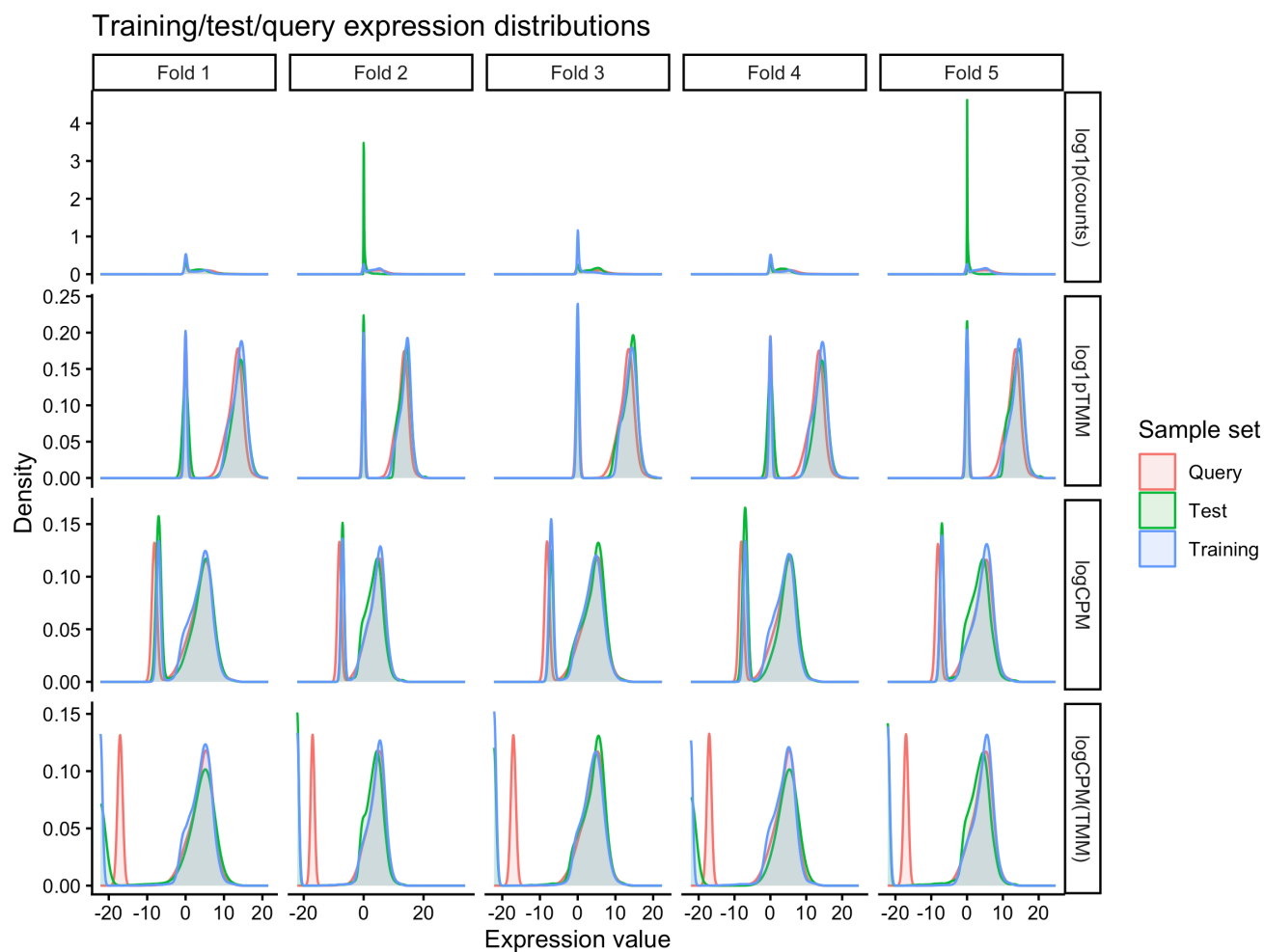
+       vals <- sample(vals, size = n_values)
+     }
+
+     tibble(split_scheme = split_scheme, fold = factor(fold, levels = seq_
+       labels = paste("Fold", seq_len(n_folds))), sample_set = sample_se
+       value = transform_fn(vals))
+   }
+ }
+
+ > set.seed(split_seed)
+
+ > labeltransfer_expr_qc_df <- imap_dfr(split_plan_list, function(plan, split
+   map2_dfr(plan$folds, seq_along(plan$folds), ~{
+     test_i <- .x
+     train_i <- setdiff(seq_len(ncol(sce_ref)), test_i)
+
+     bind_rows(sample_expression_values(sce_ref[, train_i], "counts",
+       "log1p(counts)", split_scheme, .y, transform_fn = log1p), sam
+     test_i], "counts", "Test", "log1p(counts)", split_scheme, .y),
+     sample_expression_values(sce_query, "counts", "Query", "log1p
+     split_scheme, .y, transform_fn = log1p), sample_expression
+     train_i], "logcounts", "Training", "logCPM", split_scheme
+     test_i], "logcounts", "Test", "logCPM", split_scheme, .y),
+     "logcounts", "Query", "logCPM", split_scheme, .y), sample
+     train_i], "logcounts_tmm", "Training", "log1pTMM", split_
+     .y), sample_expression_values(sce_ref[, test_i], "logcount
+     "Test", "log1pTMM", split_scheme, .y), sample_expression
+     "logcounts_tmm", "Query", "log1pTMM", split_scheme, .y),
+     train_i], "cpm_tmm_logcounts", "Training", "logCPM(TMM)",
+     .y), sample_expression_values(sce_ref[, test_i], "cpm_tmm
+     "Test", "logCPM(TMM)", split_scheme, .y), sample_expressi
+     "cpm_tmm_logcounts", "Query", "logCPM(TMM)", split_scheme
+   })

```

```

+ })
>
> qc_folds <- ggplot(labeltransfer_expr_qc_df %>%
+   filter(split_scheme == "dataset_id"), aes(x = value, fill = sample_set)
+   geom_density(alpha = 0.15) + facet_grid(assay ~ fold, scales = "free"
+   labs(x = "Expression value", y = "Density", fill = "Sample set", color = "Sample set",
+     title = "Training/test/query expression distributions")
>
> qc_folds

```



```
> ggsave("../output/gene_distribution_each_fold.pdf", width = 12, height = 8)
```

```
> lt_probs_list <- vector("list", 0)
> fold_truth_pred <- vector("list", 0)
>
> for (split_scheme in names(split_plan_list)) {
+   plan <- split_plan_list[[split_scheme]]
+
+   for (i in seq_along(plan$folds)) {
+     cat(split_scheme, "fold", i, "\n")
+
+     test_i <- plan$folds[[i]]
+     train_i <- setdiff(seq_len(ncol(seu_ref)), test_i)
+
+     seu_train <- seu_ref[, train_i]
+     seu_test <- seu_ref[, test_i]
+
+     seu_train <- NormalizeData(seu_train) %>%
+       FindVariableFeatures() %>%
+       ScaleData(verbose = FALSE) %>%
+       RunPCA(npcs = 20, verbose = FALSE)
+
+     feats <- VariableFeatures(seu_train)
+
+     seu_test <- NormalizeData(seu_test) %>%
+       ScaleData(features = feats, verbose = FALSE) %>%
+       RunPCA(features = feats, npcs = 20, verbose = FALSE)
+
+     anchors <- FindTransferAnchors(reference = seu_train, query = seu_test,
+       reference.reduction = "pca", k.anchor = 10)
+
+     predictions <- TransferData(anchorset = anchors, refdata = seu_test)
```



```

+         dims = 1:10, k.weight = 20)
+
+     truth_clean <- seu_test$ethnicity_clean
+     pred_clean <- predictions$predicted.id
+
+     fold_truth_pred[[length(fold_truth_pred) + 1]] <- tibble(split_scheme =
+       fold = i, truth = factor(truth_clean, levels = clean_levels),
+       levels = clean_levels), model = "LabelTransfer")
+
+     score_cols <- grep("^prediction\\.score\\.", colnames(predictions))
+
+     lt_prob <- predictions[, score_cols, drop = FALSE] %>%
+       as_tibble() %>%
+       rename_with(~sub("^prediction\\.score\\.", "", .x)) %>%
+       mutate(across(everything(), ~as.numeric(.x))) %>%
+       mutate(truth = factor(truth_clean, levels = clean_levels), fold =
+         model = "LabelTransfer", split_scheme = split_scheme)
+
+     missing_cols <- setdiff(clean_levels, colnames(lt_prob))
+     if (length(missing_cols) > 0) {
+       lt_prob[missing_cols] <- 0
+     }
+
+     lt_probs_list[[length(lt_probs_list) + 1]] <- lt_prob %>%
+       dplyr::select(all_of(clean_levels), truth, fold, model, split_scheme)
+   }
+ }

```

dataset_id fold 1

dataset_id fold 2

dataset_id fold 3

dataset_id fold 4

dataset_id fold 5

tissue_groups fold 1

tissue_groups fold 2

tissue_groups fold 3

tissue_groups fold 4

tissue_groups fold 5

```
> lt_model_probs <- bind_rows(lt_probs_list)
> results_all <- bind_rows(fold_truth_pred)
```

```
> fold_metrics_lt <- results_all %>%
+   group_by(split_scheme, model, fold) %>%
+   summarise(acc = mean(truth == prediction), balanced_acc = bal_accuracy(
+     prediction), macro_f1 = f_meas_vec(truth, prediction, estimator = "macro_weighted",
+     weighted_f1 = f_meas_vec(truth, prediction, estimator = "macro_weighted",
+     .groups = "drop")
+
+ summary_lt <- fold_metrics_lt %>%
+   group_by(split_scheme, model) %>%
+   summarise(acc_mean = mean(acc), acc_sd = sd(acc), balanced_acc_mean =
```

```

+       balanced_acc_sd = sd(balanced_acc), macro_f1_mean = mean(macro_f1
+       weighted_f1_mean = mean(weighted_f1), weighted_f1_sd = sd(weighte
+
+
+ summary_lt

```

```

# A tibble: 2 x 10
  split_scheme model      acc_mean acc_sd balanced_acc_mean balanced_ac
  <chr>         <chr>      <dbl> <dbl>          <dbl>          <dbl>
1 dataset_id   LabelTransfer  0.695 0.293          0.501          0.
2 tissue_groups LabelTransfer  0.643 0.0359         0.514          0.
# i 4 more variables: macro_f1_mean <dbl>, macro_f1_sd <dbl>,
#   weighted_f1_mean <dbl>, weighted_f1_sd <dbl>

```

```

> per_class_lt <- results_all %>%
+   group_by(split_scheme, model, fold) %>%
+   group_modify(~{
+     df <- .x
+     bind_rows(lapply(clean_levels, function(cls) {
+       df2 <- df %>%
+         mutate(truth_binary = factor(if_else(truth == cls, "yes",
+         levels = c("yes", "no")), pred_binary = factor(if_else(
+         cls, "yes", "no"), levels = c("yes", "no")))
+
+       prec <- precision(df2, truth_binary, pred_binary, event_level = "
+       rec <- recall(df2, truth_binary, pred_binary, event_level = "
+       f1 <- f_meas(df2, truth_binary, pred_binary, event_level = "f
+
+       tibble(class = cls, precision = ifelse(is.na(prec), 0, prec),
+         0, rec), f1 = ifelse(is.na(f1), 0, f1), support = sum(df$
+         cls))
+     })))

```

```

+   }) %>%
+   ungroup() %>%
+   mutate(Label = clean_to_ori_name[class]) %>%
+   group_by(split_scheme, model, Label) %>%
+   summarise(precision_mean = mean(precision), precision_sd = sd(precision),
+             recall_sd = sd(recall), f1_mean = mean(f1), f1_sd = sd(f1), support_total = sum(support))
+   .groups = "drop")
>
> per_class_lt

```

```

# A tibble: 10 x 10
  split_scheme model Label precision_mean precision_sd recall_mean recall_sd
  <chr>         <chr> <fct>          <dbl>          <dbl>          <dbl>      <dbl>
1 dataset_id   LabelT~ Afri~          0.0606          0.136          0.0149      0.0149
2 dataset_id   LabelT~ East~          0.00323         0.00721         0.0667      0.0667
3 dataset_id   LabelT~ Euro~          0.746           0.290          0.917      0.917
4 dataset_id   LabelT~ Hisp~          0              0              0           0
5 dataset_id   LabelT~ Sout~          0              0              0           0
6 tissue_groups LabelT~ Afri~          0.0369          0.0677          0.0758      0.0758
7 tissue_groups LabelT~ East~          0.0723          0.0760          0.169      0.169
8 tissue_groups LabelT~ Euro~          0.726           0.0507          0.864      0.864
9 tissue_groups LabelT~ Hisp~          0.0419          0.0634          0.0421      0.0421
10 tissue_groups LabelT~ Sout~          0              0              0           0
# i 3 more variables: f1_mean <dbl>, f1_sd <dbl>, support_total <int>

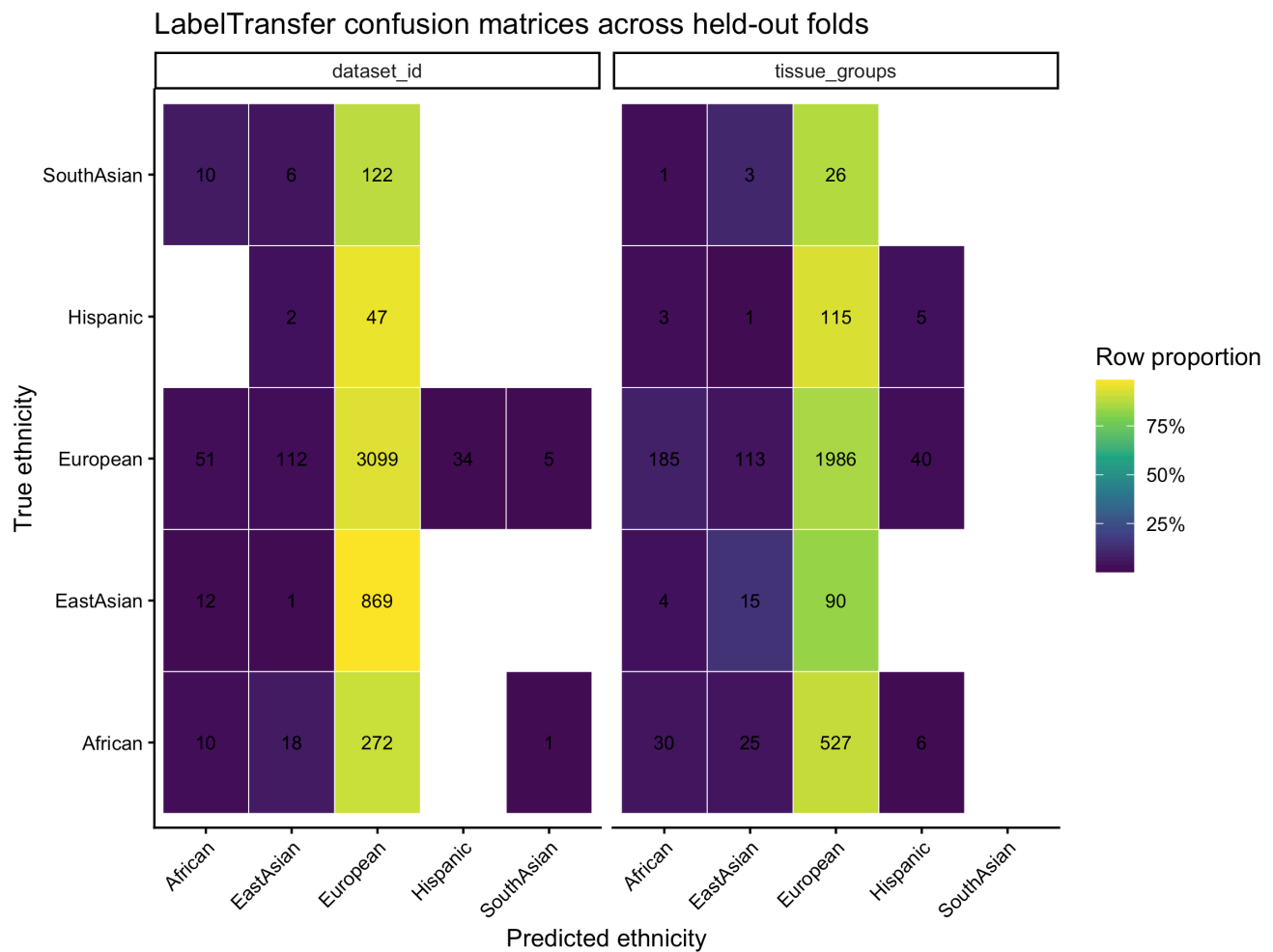
```

```

> lt_confusion_df <- results_all %>%
+   dplyr::count(split_scheme, truth, prediction, name = "n") %>%
+   group_by(split_scheme, truth) %>%
+   mutate(prop = n/sum(n)) %>%
+   ungroup() %>%
+   mutate(truth_label = clean_to_ori_name[as.character(truth)], prediction_label = clean_to_ori_name[as.character(prediction)])

```

```
>
> ggplot(lt_confusion_df, aes(x = prediction_label, y = truth_label, fill =
+   geom_tile(color = "white") + geom_text(aes(label = n), size = 3) + sc
+   facet_wrap(~split_scheme) + theme_classic() + labs(x = "Predicted ethn
+   y = "True ethnicity", fill = "Row proportion", title = "LabelTransfer
+   theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```

> lt_roc_input <- lt_model_probs %>%
+   pivot_longer(cols = all_of(clean_levels), names_to = "class", values_
+   mutate(truth_binary = factor(truth == class, levels = c(FALSE, TRUE),
+   "yes"))))
>
> lt_roc_input_ok <- lt_roc_input %>%
+   group_by(split_scheme, class, fold) %>%
+   filter(any(truth_binary == "yes") & any(truth_binary == "no")) %>%
+   ungroup()
>
> lt_roc_curves <- lt_roc_input_ok %>%
+   group_by(split_scheme, class, fold) %>%
+   roc_curve(truth_binary, prob, event_level = "second") %>%
+   ungroup() %>%
+   transmute(split_scheme = split_scheme, model_display = "LabelTransfer
+   fold = fold, fpr = 1 - specificity, tpr = sensitivity)
>
> lt_auc_by_class <- lt_roc_input_ok %>%
+   group_by(split_scheme, class, fold) %>%
+   roc_auc(truth_binary, prob, event_level = "second") %>%
+   ungroup() %>%
+   transmute(split_scheme = split_scheme, model_display = "LabelTransfer
+   fold = fold, auc = .estimate)
>
> lt_auc_by_class

```

A tibble: 50 x 5

	split_scheme	model_display	ethnicity	fold	auc
	<chr>	<chr>	<fct>	<int>	<dbl>
1	dataset_id	LabelTransfer	African	1	0.561
2	dataset_id	LabelTransfer	African	2	0.459
3	dataset_id	LabelTransfer	African	3	0.640

```

4 dataset_id    LabelTransfer African      4 0.509
5 dataset_id    LabelTransfer African      5 0.154
6 dataset_id    LabelTransfer EastAsian    1 0.334
7 dataset_id    LabelTransfer EastAsian    2 0.888
8 dataset_id    LabelTransfer EastAsian    3 0.476
9 dataset_id    LabelTransfer EastAsian    4 0.702
10 dataset_id   LabelTransfer EastAsian    5 0.461
# i 40 more rows

```

```

> roc_py <- read_csv("../roc_points.csv") %>%
+   mutate(model_display = model) %>%
+   mutate(model_display = str_remove(model_display, "_S_")) %>%
+   filter(model_display %in% selected_models) %>%
+   transmute(split_scheme = split_scheme, model_display = model_display,
+             fold = fold, fpr = fpr, tpr = tpr)
>
> auc_py <- read_csv("../auc_by_class.csv") %>%
+   mutate(model_display = model) %>%
+   mutate(model_display = str_remove(model_display, "_S_")) %>%
+   filter(model_display %in% selected_models) %>%
+   transmute(split_scheme = split_scheme, model_display = model_display,
+             fold = fold, auc = auc)
>
> roc_all <- bind_rows(roc_py, lt_roc_curves) %>%
+   mutate(split_scheme = factor(split_scheme, levels = c("dataset_id", "
+             model_display = factor(model_display), ethnicity = factor(ethnicity)
+   mutate(model_display = str_remove(model_display, "_S_"))
>
> auc_all <- bind_rows(auc_py, lt_auc_by_class) %>%
+   mutate(split_scheme = factor(split_scheme, levels = c("dataset_id", "
+             model_display = factor(model_display), ethnicity = factor(ethnicity)
+   mutate(model_display = str_remove(model_display, "_S_"))

```

```

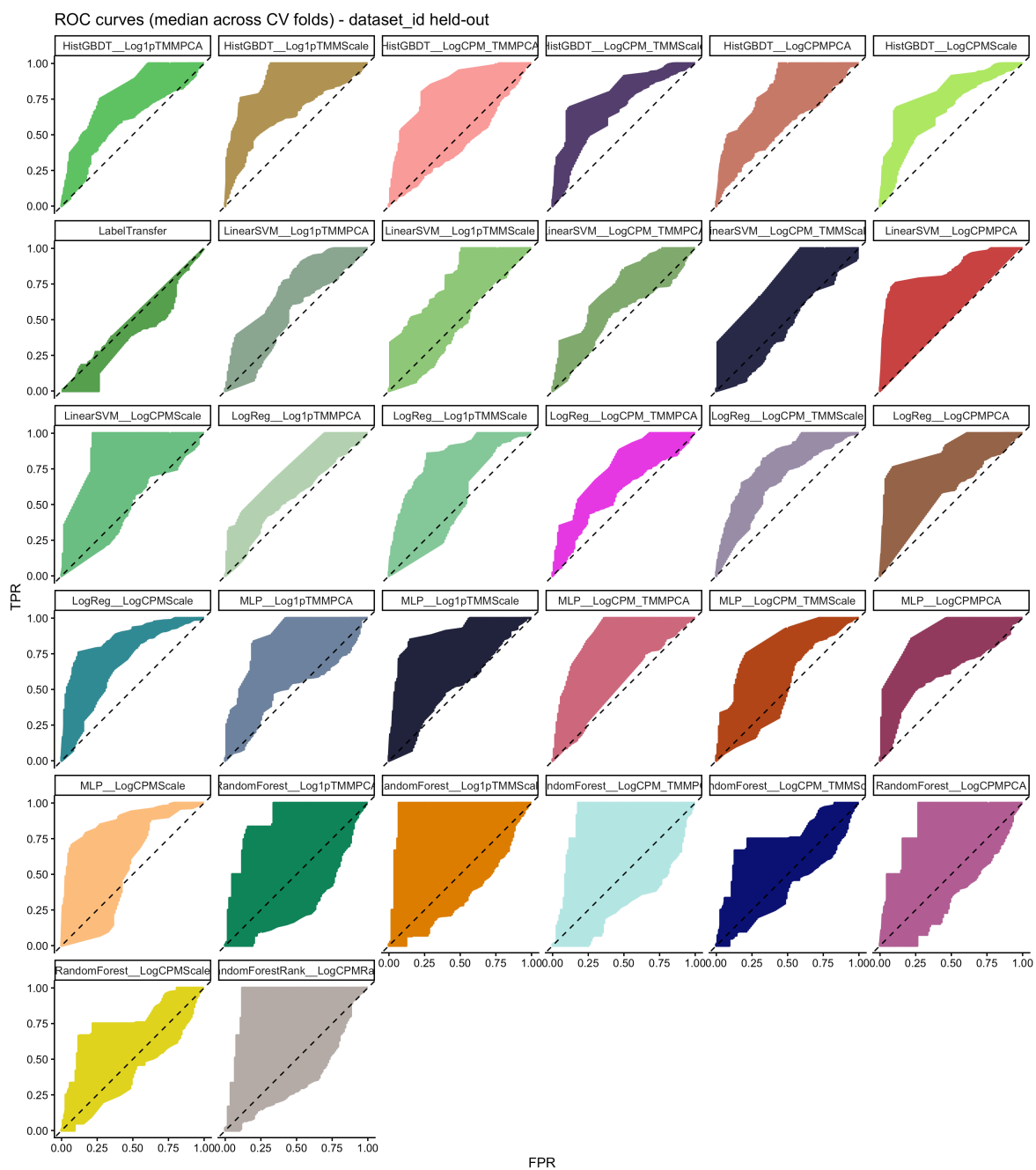
> method_names <- unique(as.character(roc_all$model_display))
>
> if (length(method_names) <= length(ning_colors)) {
+   method_cols <- ning_colors[seq_along(method_names)]
+ } else {
+   method_cols <- colorRampPalette(ning_colors)(length(method_names))
+ }
>
> names(method_cols) <- method_names
>
> fpr_grid <- seq(0, 1, by = 0.01)
>
> interp_roc <- function(df_one, grid = fpr_grid) {
+   df_one <- df_one %>%
+     arrange(fpr, tpr) %>%
+     group_by(fpr) %>%
+     summarise(tpr = max(tpr), .groups = "drop")
+
+   tibble(fpr = grid, tpr = approx(df_one$fpr, df_one$tpr, xout = grid,
+     rule = 2)$y)
+ }
>
> roc_one_line <- roc_all %>%
+   group_by(split_scheme, model_display, ethnicity, fold) %>%
+   group_modify(~interp_roc(.x)) %>%
+   ungroup() %>%
+   group_by(split_scheme, model_display, ethnicity, fpr) %>%
+   summarise(tpr = median(tpr, na.rm = TRUE), .groups = "drop")

> ggplot(roc_one_line %>%
+   filter(split_scheme == "dataset_id"), aes(x = fpr, y = tpr, color = m
+   geom_line(linewidth = 1) + facet_wrap(~model_display) + geom_abline(1

```



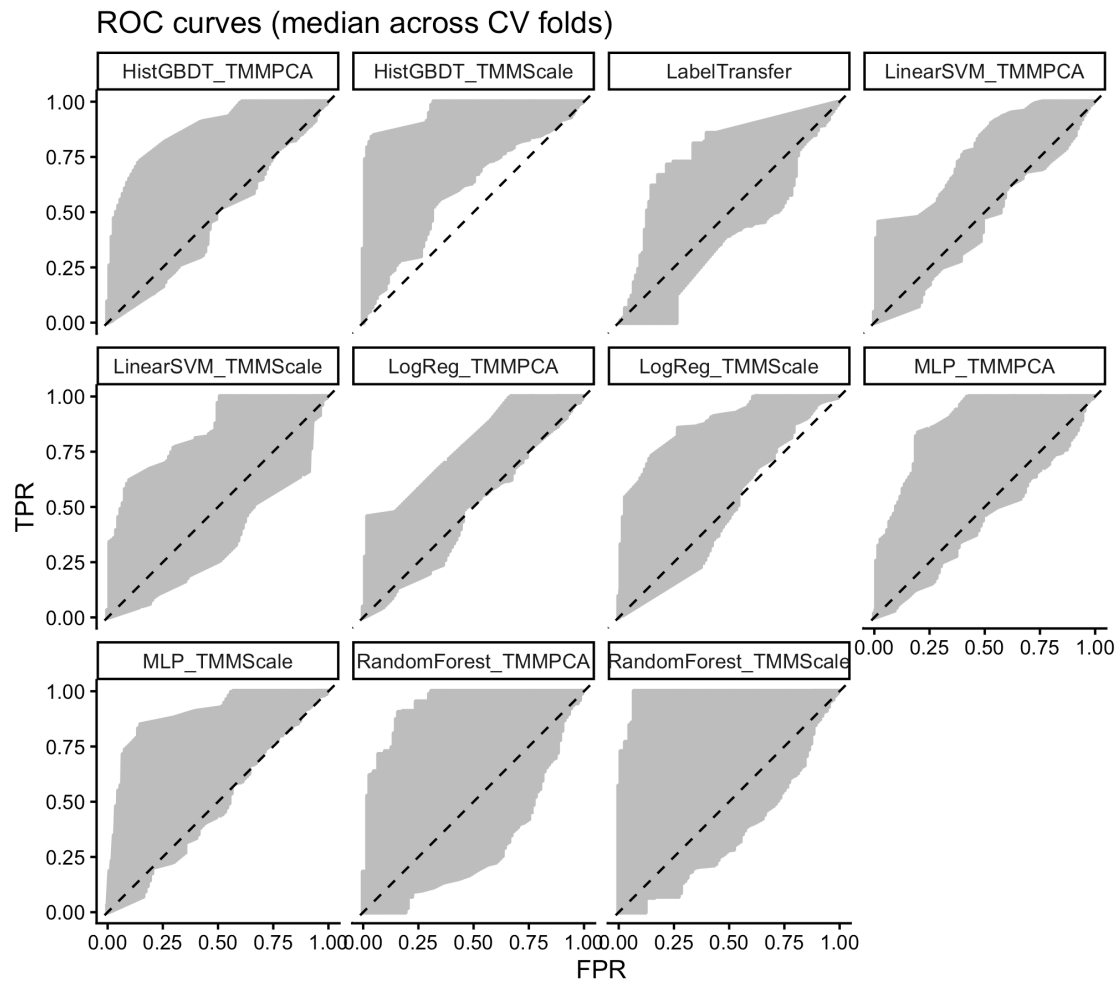
```
+ scale_color_manual(values = method_cols) + coord_equal() + theme_classic()
+ labs(x = "FPR", y = "TPR", color = "Method", title = "ROC curves (method)")
+ theme(legend.position = "none")
```



```

> roc_one_line_final <- roc_one_line %>%
+   filter(!grepl("CPM", model_display))
>
> method_cols_f <- method_cols[unique(roc_one_line_final$model_display)]
>
> roc_one_line_final <- roc_one_line_final %>%
+   mutate(model_display = str_remove(model_display, "_Log1p"))
>
> names(method_cols_f) <- unique(roc_one_line_final$model_display)
>
> ggplot(roc_one_line_final, aes(x = fpr, y = tpr, color = model_display)) +
+   color = "gray") + facet_wrap(~model_display) + geom_abline(linetype =
+   coord_equal() + theme_classic() + labs(x = "FPR", y = "TPR", color =
+   title = "ROC curves (median across CV folds)") + theme(legend.position

```



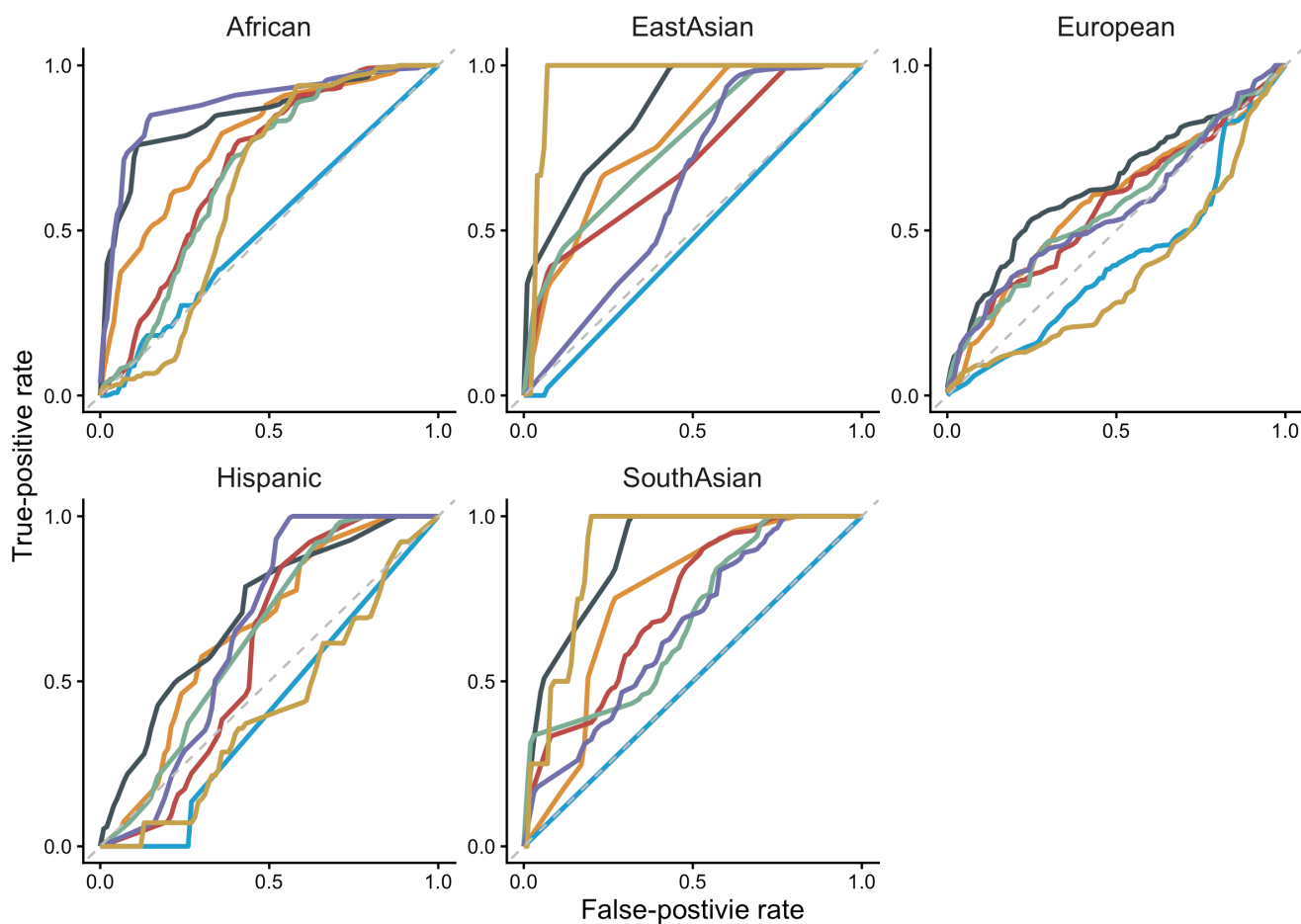
```
> ggsave("../output/ROC_all.png", width = 10, height = 10, bg = "white")
```

```
> good_models <- c("HistGBDT_TMMScale", "HistGBDT_TMMPCA", "LabelTransfer",
+   "LogReg_TMMPCA", "MLP_TMMScale", "RandomForest_TMMScale")
>
> gm_cols <- c("#44545B", "#DE903F", "#249FD1", "#BB4E49", "#7EAE94", "#7570B3")
>
> names(gm_cols) <- good_models
```

```

> p_model_performance <- ggplot(roc_one_line_final %>%
+   filter(split_scheme == "dataset_id") %>%
+   filter(model_display %in% good_models), aes(x = fpr, y = tpr, color =
+   geom_line(linewidth = 1) + facet_wrap(~ethnicity, nrow = 2, scales =
+   geom_abline(linetype = "dashed", color = "gray") + scale_color_manual
+   theme_classic() + labs(x = "False-positive rate", y = "True-positive r
+   title = "") + scale_x_continuous(breaks = c(0, 0.5, 1)) + scale_y_con
+   0.5, 1)) + theme(legend.position = "none", strip.background = element
+   strip.text = element_text(size = 12), axis.title = element_text(size :
+
>
> p_model_performance

```



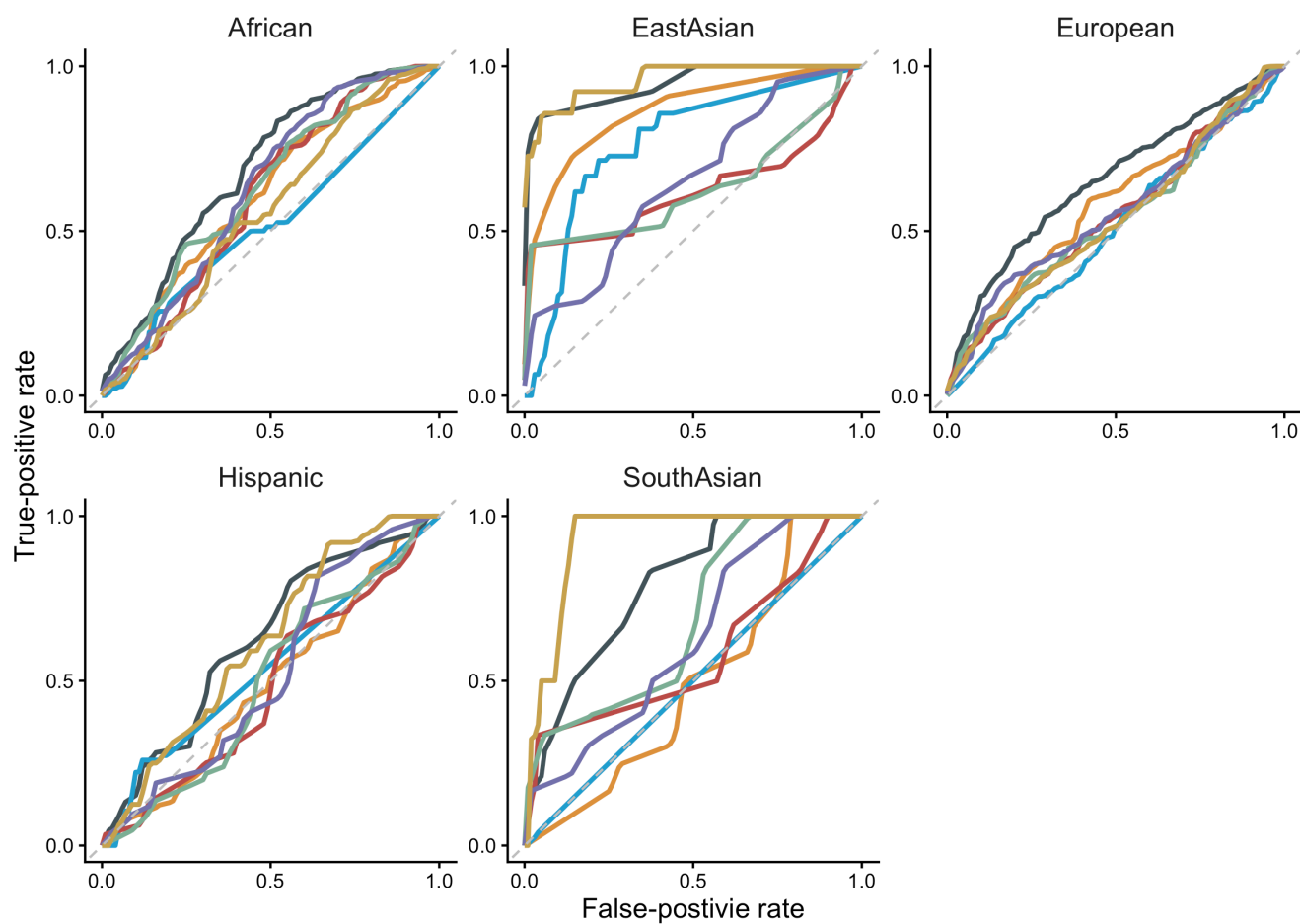
```
> ggsave("../output/roc_performance.pdf", width = 7, height = 5, bg = "white")
```

```
> p_model_performance_tho <- ggplot(roc_one_line_final %>%
+   filter(split_scheme == "tissue_groups") %>%
+   filter(model_display %in% good_models), aes(x = fpr, y = tpr, color =
+   geom_line(linewidth = 1) + facet_wrap(~ethnicity, nrow = 2, scales =
+   geom_abline(linetype = "dashed", color = "gray") + scale_color_manual
+   theme_classic() + labs(x = "False-postivie rate", y = "True-positive
+   title = "") + scale_x_continuous(breaks = c(0, 0.5, 1)) + scale_y_con
```

```

+   0.5, 1)) + theme(legend.position = "none", strip.background = element
+   strip.text = element_text(size = 12), axis.title = element_text(size =
+
>
> p_model_performance_tho

```



```

> ggsave("../output/roc_performance_tho.pdf", width = 13, height = 7, bg =

```

```

> overall_py2 <- read_csv("../overall_performance.csv") %>%
+   mutate(model = str_remove(model, "_S_")) %>%
+   rename_with(~gsub("_std$", "_sd", .x)) %>%
+   mutate(model_display = model) %>%
+   filter(model_display %in% selected_models)
>
>
> pereth_py2 <- read_csv("../pereth_performance.csv") %>%
+   mutate(model = str_remove(model, "_S_")) %>%
+   mutate(model_display = model, Label = as.character(ethnicity)) %>%
+   filter(model_display %in% selected_models) %>%
+   dplyr::select(-model, -ethnicity)
>
> overall_lt2 <- summary_lt %>%
+   transmute(split_scheme = split_scheme, model_display = "LabelTransfer
+     acc_sd = acc_sd, balanced_acc_mean = balanced_acc_mean, balanced_
+     macro_f1_mean = macro_f1_mean, macro_f1_sd = macro_f1_sd, weighte
+     weighted_f1_sd = weighted_f1_sd)
>
> pereth_lt2 <- per_class_lt %>%
+   transmute(split_scheme = split_scheme, model_display = "LabelTransfer
+     precision_mean = precision_mean, precision_sd = precision_sd, rec
+     recall_sd = recall_sd, f1_mean = f1_mean, f1_sd = f1_sd, support_
>
> overall_all <- bind_rows(overall_py2 %>%
+   dplyr::select(-model), overall_lt2) %>%
+   mutate(split_scheme = factor(split_scheme, levels = c("dataset_id", "
+     model_display = factor(model_display, levels = c(selected_models,
>
> pereth_all <- bind_rows(pereth_py2, pereth_lt2) %>%
+   mutate(split_scheme = factor(split_scheme, levels = c("dataset_id", "
+     model_display = factor(model_display, levels = c(selected_models,

```

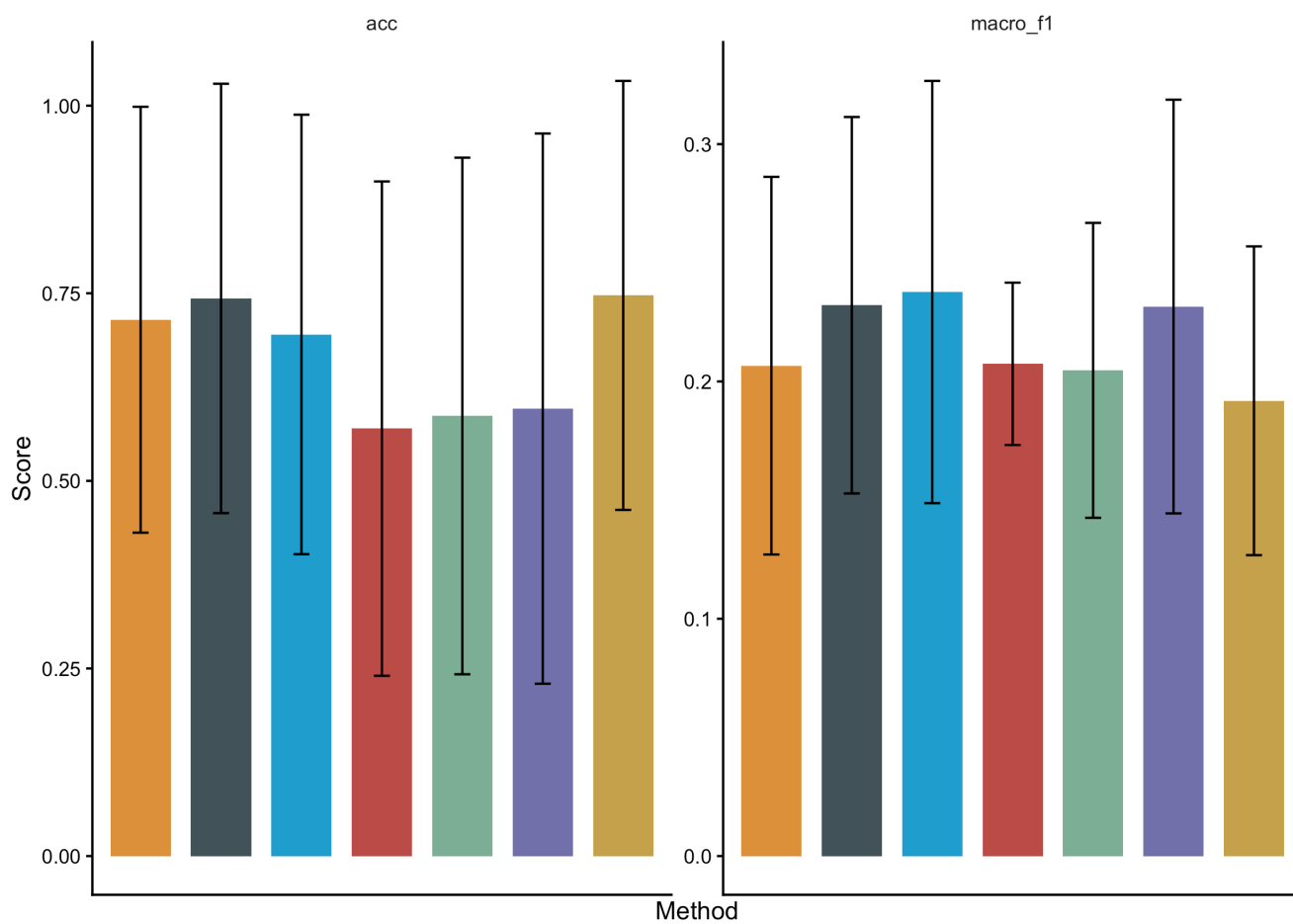


```

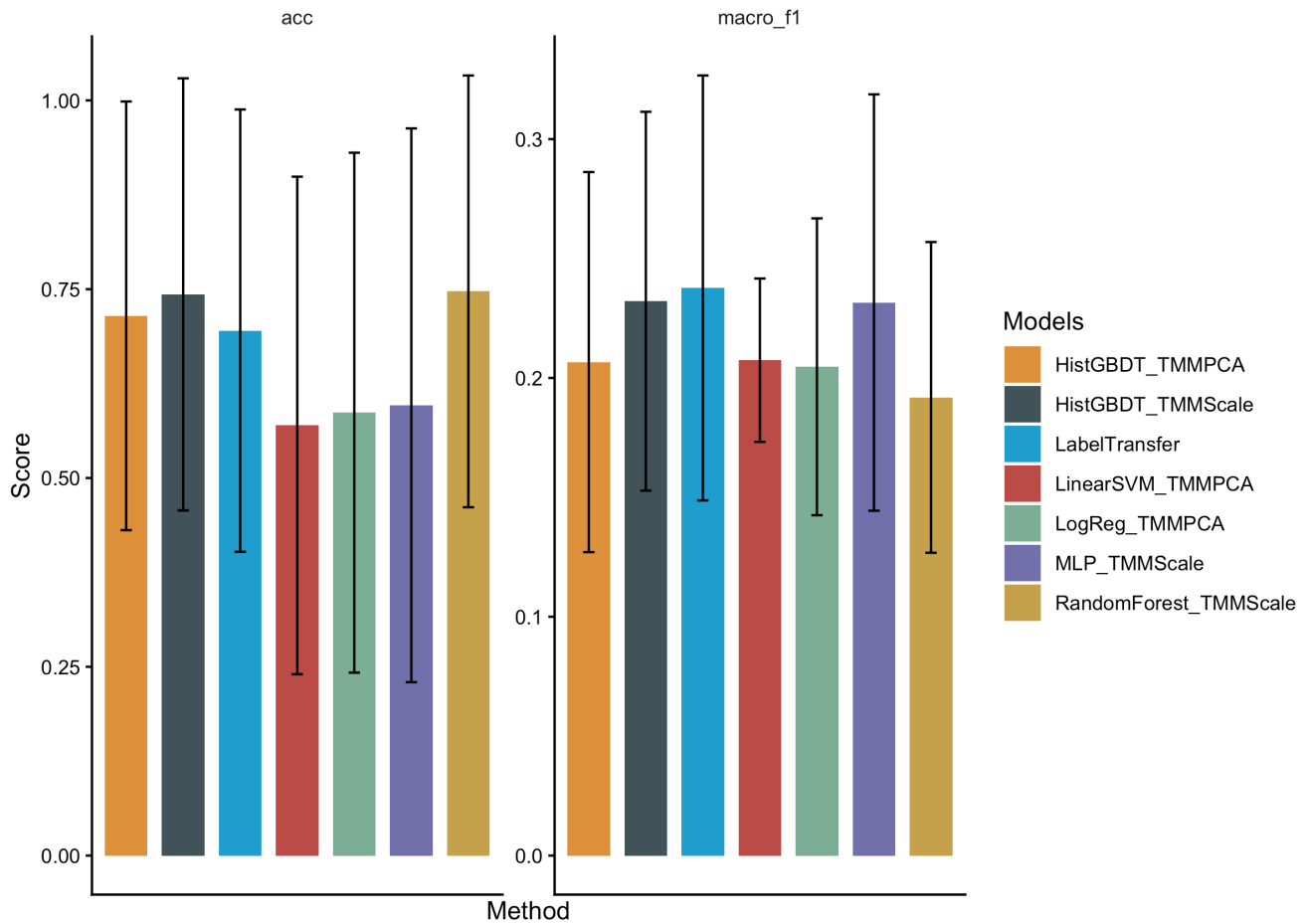
+   mutate(model_display = str_remove(model_display, "_Log1p"))

> overall_long <- overall_all %>%
+   pivot_longer(cols = ends_with("_mean"), names_to = "metric", values_to = "mean") %>%
+   mutate(metric = sub("_mean$", "", metric)) %>%
+   left_join(overall_all %>%
+     pivot_longer(cols = ends_with("_sd"), names_to = "metric_sd", values_to = "sd") %>%
+     mutate(metric_sd = sub("_sd$", "", metric_sd)) %>%
+     dplyr::select(split_scheme, model_display, metric, sd), by = c("split_scheme",
+       "model_display", "metric")) %>%
+   filter(!metric %in% c("balanced_acc", "weighted_f1")) %>%
+   mutate(model_display = str_remove(model_display, "_Log1p"))
>
> ov_performance <- ggplot(overall_long %>%
+   filter(split_scheme == "dataset_id", model_display %in% good_models),
+   aes(y = mean, fill = model_display)) + geom_col(width = 0.75) + geom_errorbar(aes(
+     y_min = mean - sd, y_max = mean + sd), width = 0.2) + facet_wrap(~metric, scales = "free")
+   scale_fill_manual(values = gm_cols) + guides(fill = guide_legend(override.aes = FALSE))
+   labs(fill = "Models") + theme_classic() + theme(axis.text.x = element_text(angle = 45),
+     axis.ticks.x = element_blank(), legend.position = "none", strip.background = "white",
+     labs(x = "Method", y = "Score", title = ""))
>
> ov_performance

```



```
> ggsave("../output/benchmark_overall_performance.pdf", width = 7, height = 7)
>
> ov_performance + theme(legend.position = "right") + guides(fill = guide_legend())
```



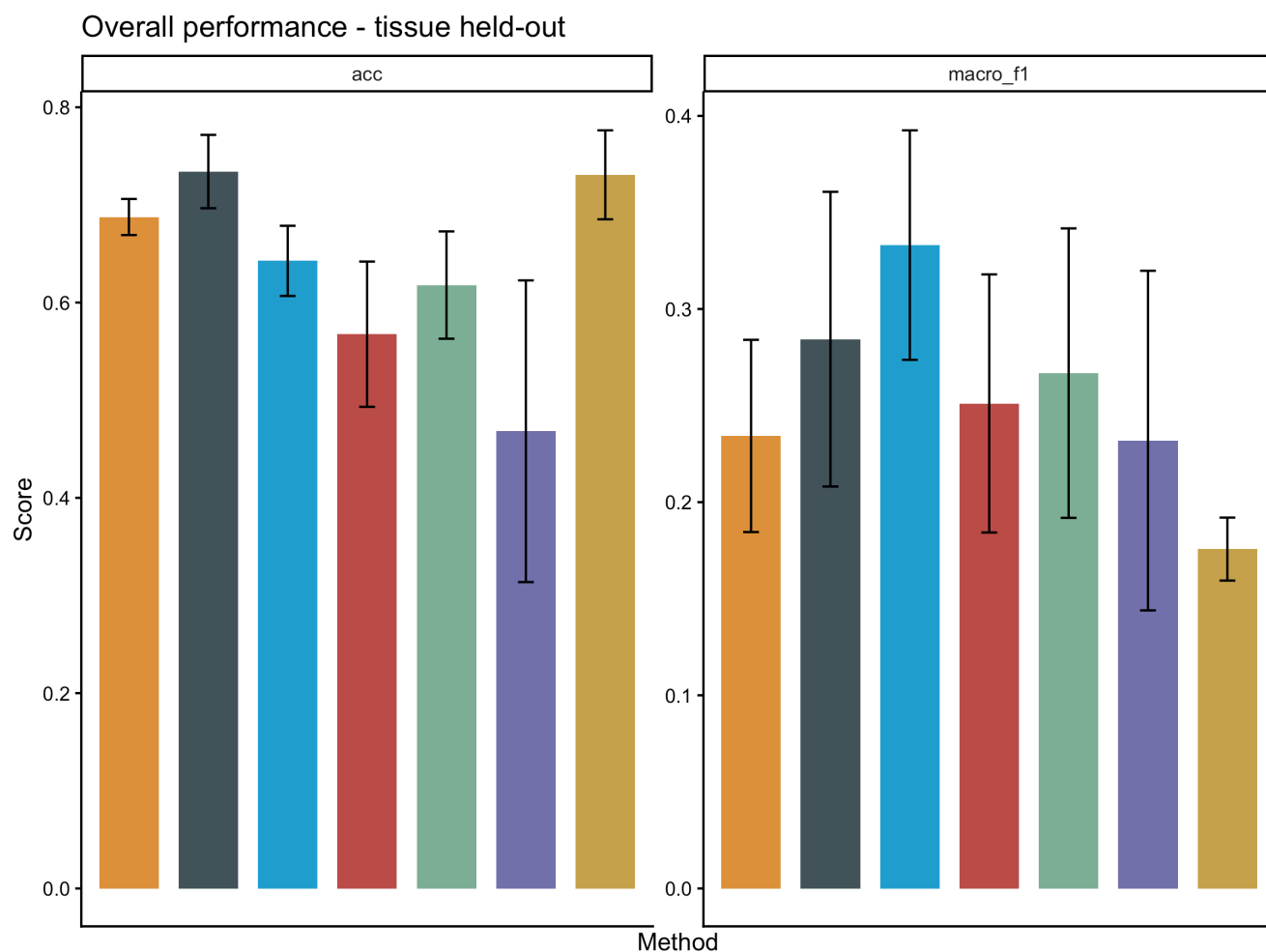
```
> ggsave("../output/benchmark_overall_performance_legend.pdf", width = 7, height = 7,
+         bg = "white")
```

```
> ov_performance_tho <- ggplot(overall_long %>%
+   filter(split_scheme == "tissue_groups", model_display %in% good_model_names) +
+   y = mean, fill = model_display)) + geom_col(width = 0.75) + geom_errorbar(
+   sd, ymax = mean + sd), width = 0.2) + facet_wrap(~metric, scales = "free") +
+   scale_fill_manual(values = gm_cols) + theme_classic() + theme(axis.text.x =
+   axis.ticks.x = element_blank(), legend.position = "none") + labs(x = "Method",
```

```

+   y = "Score", title = "Overall performance - tissue held-out")
>
> ov_performance_tho

```



```

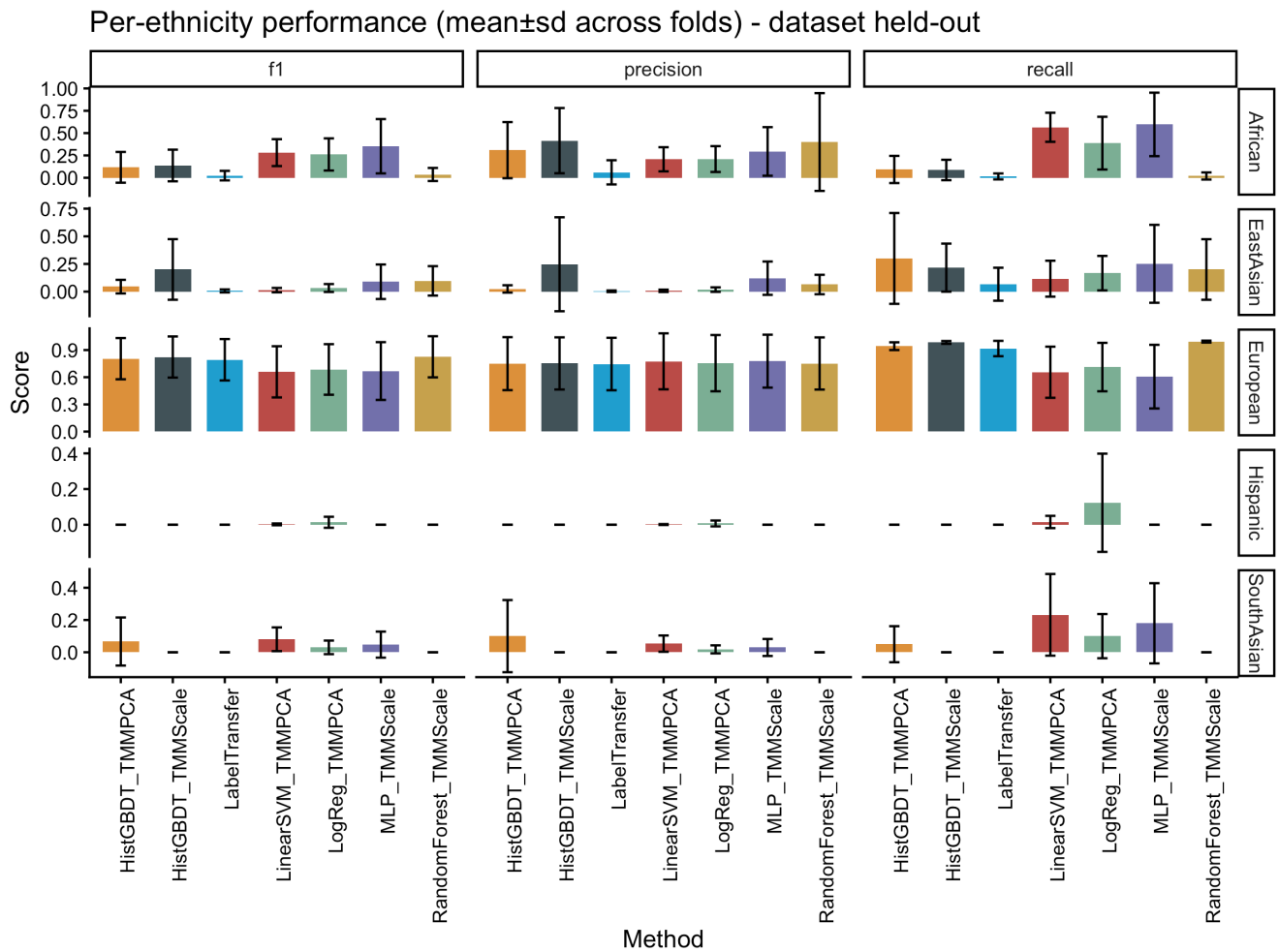
> ggsave("../output/benchmark_overall_performance_tho.pdf", width = 8, height = 6,
+   bg = "white")

```

```

> pereth_long <- pereth_all %>%
+   pivot_longer(cols = c(precision_mean, recall_mean, f1_mean), names_to =
+     values_to = "mean") %>%
+   mutate(metric = sub("_mean$", "", metric)) %>%
+   left_join(pereth_all %>%
+     pivot_longer(cols = c(precision_sd, recall_sd, f1_sd), names_to =
+       values_to = "sd") %>%
+     mutate(metric = sub("_sd$", "", metric_sd)) %>%
+     dplyr::select(split_scheme, model_display, Label, metric, sd), by =
+       "model_display", "Label", "metric"))
>
> pppho1 <- ggplot(pereth_long %>%
+   filter(split_scheme == "dataset_id", model_display %in% good_models),
+   y = mean, fill = model_display)) + geom_col(position = position_dodge
+   width = 0.7) + geom_errorbar(aes(ymin = mean - sd, ymax = mean + sd),
+   width = 0.2) + facet_grid(Label ~ metric, scales = "free_y") + scale_y
+   theme_classic() + theme(axis.text.x = element_text(angle = 90, hjust =
+   labs(x = "Method", y = "Score", title = "Per-ethnicity performance (m
>
> pppho1

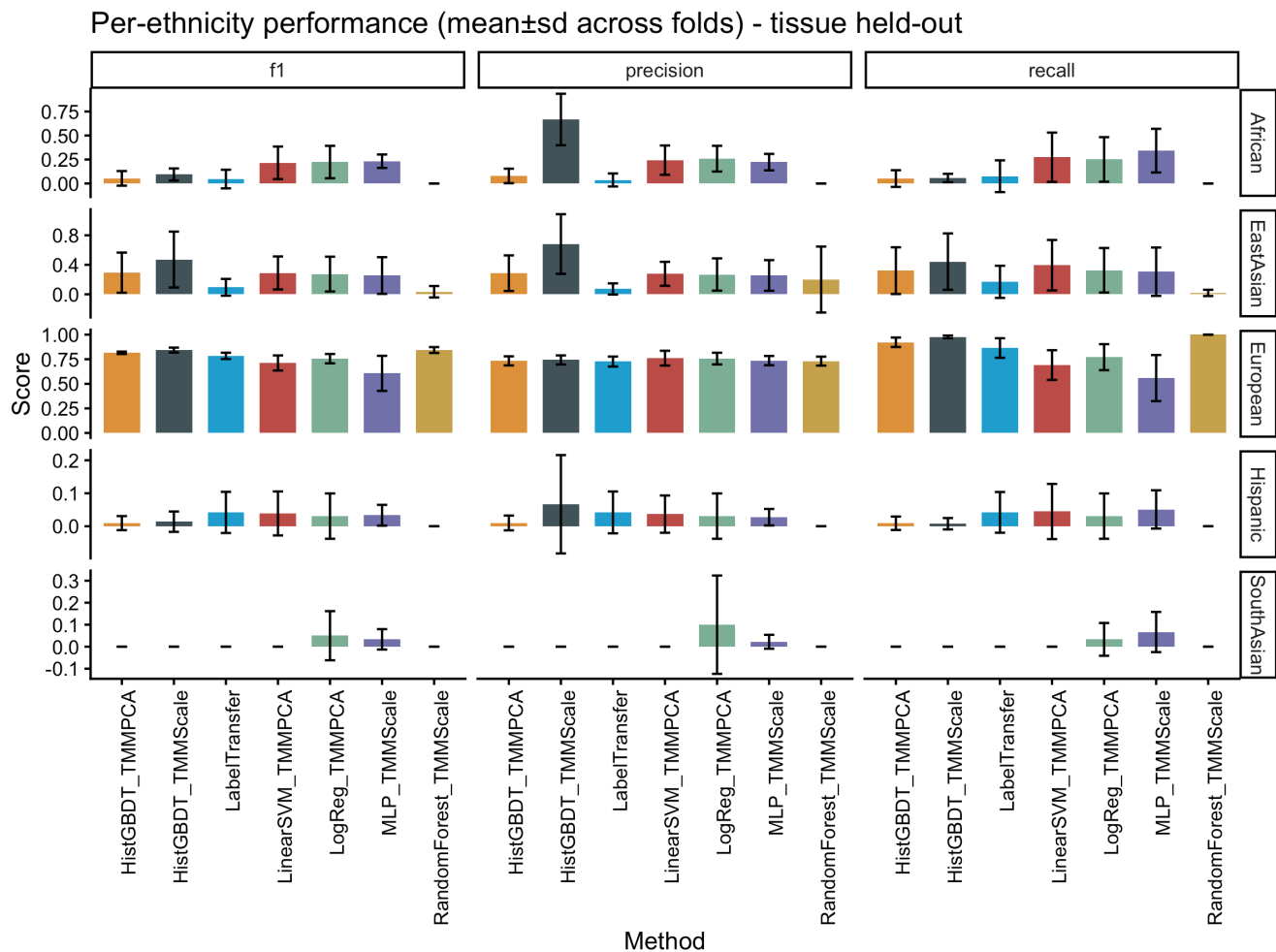
```



```
> ggsave("../output/benchmark_pereth_performance.png", width = 12, height = 10)
```

```
> pppho2 <- ggplot(pereth_long %>%
+   filter(split_scheme == "tissue_groups", model_display %in% good_model)
+   y = mean, fill = model_display)) + geom_col(position = position_dodge
+   width = 0.7) + geom_errorbar(aes(ymin = mean - sd, ymax = mean + sd),
+   width = 0.2) + facet_grid(Label ~ metric, scales = "free_y") + scale_y
+   theme_classic() + theme(axis.text.x = element_text(angle = 90, hjust = 1)
+   labs(x = "Method", y = "Score", title = "Per-ethnicity performance (mean±sd across folds) - dataset held-out")
```

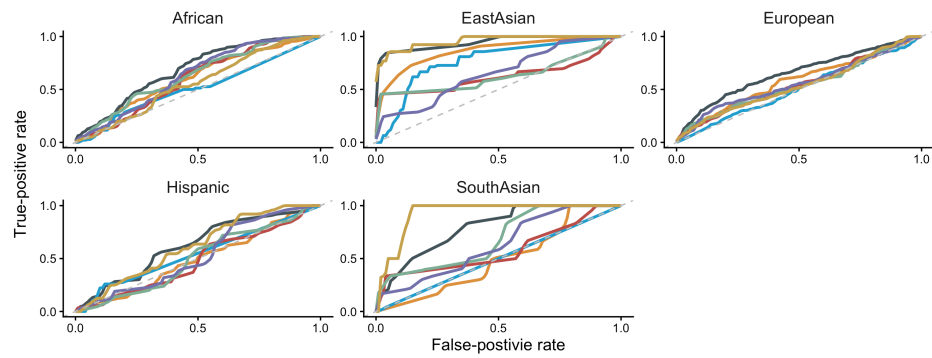
```
>
> pppho2
```



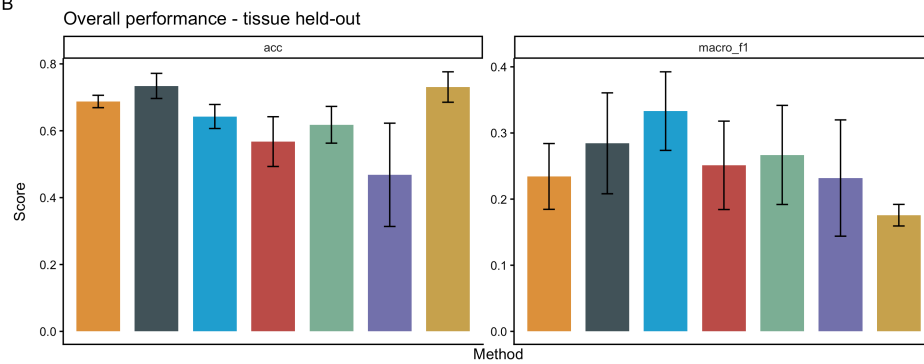
```
> ggsave("../output/benchmark_pereth_performance_tho.pdf", width = 12, height = 12,
+         bg = "white")
```

```
> p_model_performance_tho + ov_performance_tho + pppho2 + plot_layout(ncol = 2, 3)) +
+   plot_annotation(tag_levels = "A")
```

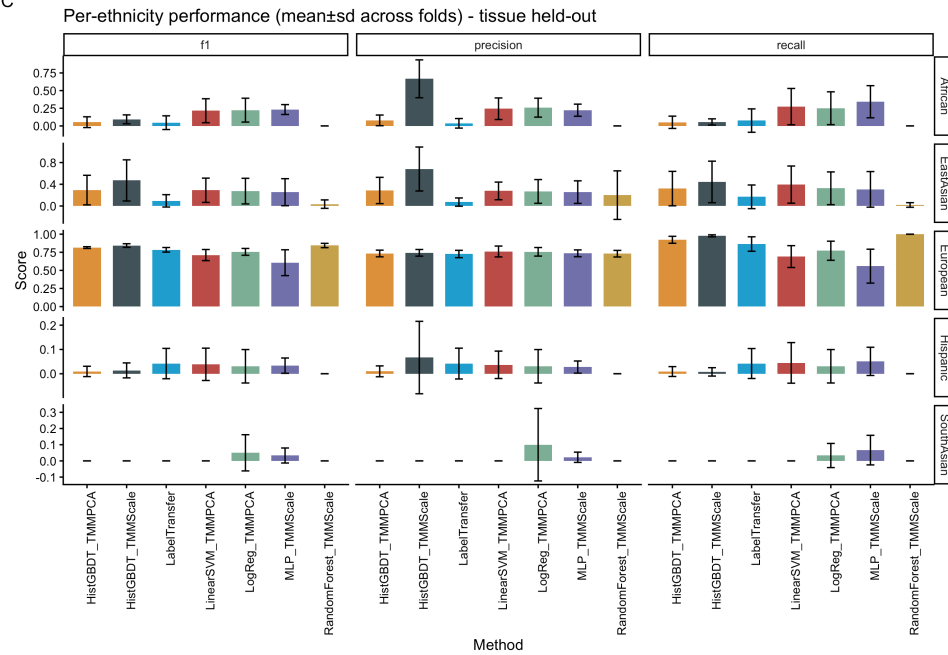
A



B

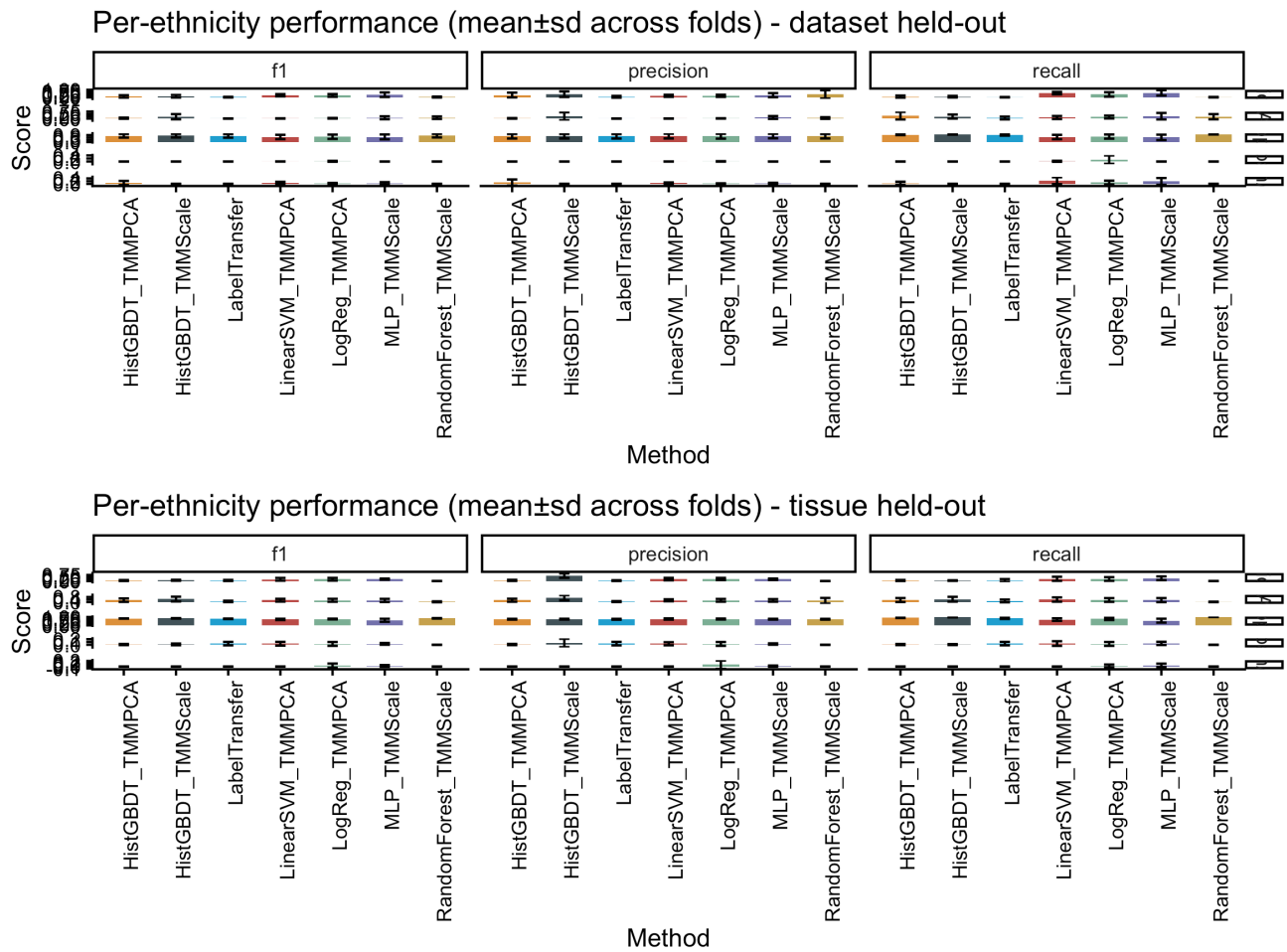


C




```
> ggsave("../output/benchmark_tho.png", width = 10, height = 15, bg = "white")
```

```
> pppho1 + pppho2 + plot_layout(ncol = 1)
```



```
> ggsave("../output/benchmark_models_per_ethnicity.png", width = 10, height
```

7 Final prediction results

```
> final_predictions <- read_csv("../histgbdt_Log1pTMM_S_PCA_query_prediction")
>
> table(final_predictions$predicted_ethnicity)
```

African	EastAsian	European	Hispanic	SouthAsian
30	73	1881	26	1

8 Markers

```
> library(org.Hs.eg.db)
>
> df <- assay(sce_ref, "counts_adjusted_ethnicity") %>%
+   as.data.frame() %>%
+   rownames_to_column("gene") %>%
+   pivot_longer(-gene, names_to = "sampleid", values_to = "counts") %>%
+   left_join(colData(sce_ref) %>%
+     as.data.frame() %>%
+     dplyr::select("ethnicity_relabel") %>%
+     rownames_to_column("sampleid")) %>%
+   dplyr::select(c("ethnicity_relabel", "gene", "counts")) %>%
+   group_by(ethnicity_relabel, gene) %>%
+   summarise(meanc = mean(counts), .groups = "drop") %>%
+   pivot_wider(names_from = "gene", values_from = "meanc") %>%
+   column_to_rownames("ethnicity_relabel")
>
```

```

> marker_gene_lookup <- tibble(gene = unique(top_markers_all$gene)) %>%
+   mutate(symbol = NA_character_)
>
> ensng_idx <- str_detect(marker_gene_lookup$gene, "^ENSG")
>
> marker_gene_lookup$symbol[ensng_idx] <- mapIds(org.Hs.eg.db, keys = marker_gene_lookup$gene,
+   column = "SYMBOL", keytype = "ENSEMBL", multiVals = "first") %>%
+   unname()
>
> marker_gene_lookup <- marker_gene_lookup %>%
+   mutate(display_gene = coalesce(symbol, gene))
>
> marker_top5_table <- top_markers_all %>%
+   left_join(marker_gene_lookup, by = "gene") %>%
+   mutate(ethnicity_relabel = recode(ethnicity, Europeans = "European",
+   .default = ethnicity)) %>%
+   filter(ethnicity_relabel %in% rownames(df), !is.na(display_gene), !str_detect(gene,
+   "^ENSG")) %>%
+   arrange(ethnicity_relabel, desc(dist0_CI), desc(estimate)) %>%
+   distinct(ethnicity_relabel, display_gene, .keep_all = TRUE) %>%
+   group_by(ethnicity_relabel) %>%
+   slice_head(n = 5) %>%
+   ungroup() %>%
+   dplyr::select(ethnicity_relabel, gene, display_gene, dist0_CI, estimate,
+   ci_upper, star)
>
> marker_plot_lookup <- marker_gene_lookup %>%
+   distinct(gene, display_gene) %>%
+   mutate(plot_label = make.unique(display_gene, sep = "_"))
>
> marker_top5_labels <- marker_top5_table %>%
+   distinct(gene, display_gene) %>%

```

```

+   left_join(marker_plot_lookup, by = c("gene", "display_gene")) %>%
+   pull(plot_label)
>
> marker_genes <- marker_plot_lookup$gene
> sub_mat <- as.matrix(df[, marker_genes, drop = FALSE])
> colnames(sub_mat) <- marker_plot_lookup$plot_label[match(colnames(sub_mat),
>
> keep_cols <- apply(sub_mat, 2, function(x) {
+   all(is.finite(x)) && sd(x) > 0
+ })
>
> dropped_marker_genes <- colnames(sub_mat)[!keep_cols]
>
> sub_mat <- sub_mat[, keep_cols, drop = FALSE]
> if (ncol(sub_mat) == 0) {
+   stop("No finite, non-constant marker genes remained for the heatmap.")
+ }
> sub_mat_z <- scale(sub_mat)
>
> marker_top5_table

```

A tibble: 25 x 8

	ethnicity_relabel	gene	display_gene	dist0_CI	estimate	ci_lower	ci_upper
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	African	ENSG000000~	SFRP1	1.01	2.95	1.01	
2	African	ENSG000000~	MYL2	0.840	2.31	0.840	
3	African	ENSG000000~	ANKRD1	0.732	2.64	0.732	
4	African	ENSG000000~	MYH7	0.571	7.71	0.571	
5	African	ENSG000000~	SPINK5	0.558	2.02	0.558	
6	EastAsian	ENSG000000~	ATP6V0A4	2.20	4.15	2.20	
7	EastAsian	ENSG000000~	DNAH10	1.67	3.61	1.67	
8	EastAsian	ENSG000000~	RFPL2	0.852	2.81	0.852	

```

9 EastAsian          ENSG000000~ TRAV5          0.848      2.71      0.848
10 EastAsian          ENSG000000~ TAS2R31         0.779      2.08      0.779
# i 15 more rows
# i 1 more variable: star <chr>

```

```

> if (length(dropped_marker_genes) > 0) {
+   tibble(dropped_marker_gene = dropped_marker_genes)
+ }
>
> write_csv(marker_top5_table, "../output/top5_markers_by_ethnicity_dist0CI
>
> cluster_cols <- my_colors

```

```

> library(ComplexHeatmap)
> library(circlize)
> library(grid)
> library(scico)
>
> m <- sub_mat_z
>
> genes_selected <- c("MMP8", "LGALS2", "GJB2", "IGLV8-61")
>
> genes_selected_present <- intersect(genes_selected, colnames(m))
> marker_top5_labels_present <- intersect(marker_top5_labels, colnames(m))
> genes_to_mark <- unique(c(marker_top5_labels_present, genes_selected_present))
> mark_idx <- which(colnames(m) %in% genes_to_mark)
> mark_lbl <- colnames(m)[mark_idx]
> mark_lbl_rendered <- ifelse(mark_lbl %in% genes_selected_present, paste0(
+   mark_lbl, "</span>"), mark_lbl)
> mark_lbl_rendered <- gt_render(mark_lbl_rendered)
>
> if (length(setdiff(genes_selected, genes_selected_present)) > 0) {

```

```

+     tibble(missing_highlight_gene = setdiff(genes_selected, genes_selected))
+   }
+ }
>
> col_fun <- colorRamp2(c(min(m, na.rm = TRUE), 0, max(m, na.rm = TRUE)), s
+   palette = "vik"))
>
> row_name_cols <- cluster_cols[rownames(m)]
> row_name_cols[is.na(row_name_cols)] <- "black"
>
> ta <- HeatmapAnnotation(mark = anno_mark(at = mark_idx, side = "bottom",
+   labels_gp = gpar(fontsize = 9, fontface = "bold"), link_gp = gpar(col
+   link_width = unit(4, "mm"), link_height = unit(3, "mm")), annotation_
+   show_annotation_name = FALSE)
>
> cht <- Heatmap(m, name = "Z", col = col_fun, cluster_rows = TRUE, cluster
+   show_row_names = TRUE, show_column_names = FALSE, row_names_gp = gpar
+   col = row_name_cols), heatmap_width = unit(35, "cm"), heatmap_hei
+   "cm"), column_dend_height = unit(2.5, "cm"), row_dend_width = uni
+   border = TRUE, bottom_annotation = ta)
>
> pdf("../output/Ethnicity_markerG_heatmap_full_ComplexHeatmap.pdf", width :
> draw(cht, heatmap_legend_side = "right", annotation_legend_side = "right"
> dev.off()

```

quartz_off_screen

2

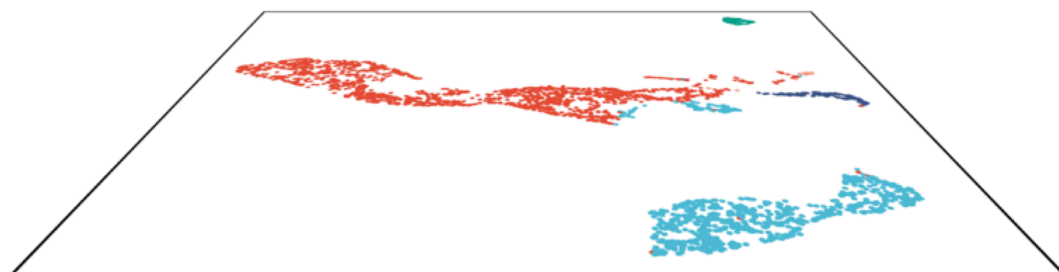
9 Merge figure

```
> library(cowplot)
> library(magick)
>
> W    <- 800
> H    <- 600
> Wt   <- 400
>
> p <- p_onlyhighscore
>
> img <- image_graph(width = W, height = H, res = 96)
> print(p)  # your ggplot
> dev.off()
```

quartz_off_screen
2

```
> ht    <- (W - Wt) / 2
> y_top <- 250          # e.g. 50px down
> H2    <- 450          # bottom edge at 450px instead of 600px
>
> coords_shrunk <- c(
+   0, 0,      ht,      y_top,
+   W, 0,      W - ht,  y_top,
+   0, H,      0,      H2,
+   W, H,      W,      H2
+ )
>
> img_trap <- image_distort(
```

```
+ img,  
+ distortion = "perspective",  
+ coordinates = coords_shrunk,  
+ bestfit     = TRUE  
+ )  
>  
>  
> plot(img_trap)
```




```

> g <- rasterGrob(
+   img_trap,
+   width = unit(1, "npc"),
+   height = unit(1, "npc"),
+   interpolate = TRUE
+ ) # keeps high fidelity :contentReference[oaicite:6]{index=6}
>
>
> p_onlyhighscore_trap <- ggdraw() +
+   draw_grob(
+     g,
+     x      = 0,
+     y      = 0.2,      # leave 20% margin below
+     width  = 1,
+     height = 0.5       # fill 60% of canvas height :contentReference[oaicite:6]{index=6}
+   ) +
+   theme(plot.margin = unit(c(0,0,0,0), "pt"))

```

```

> p <- p_query_umap
>
> img <- image_graph(width = W, height = H, res = 96)
> print(p)
> dev.off()

```

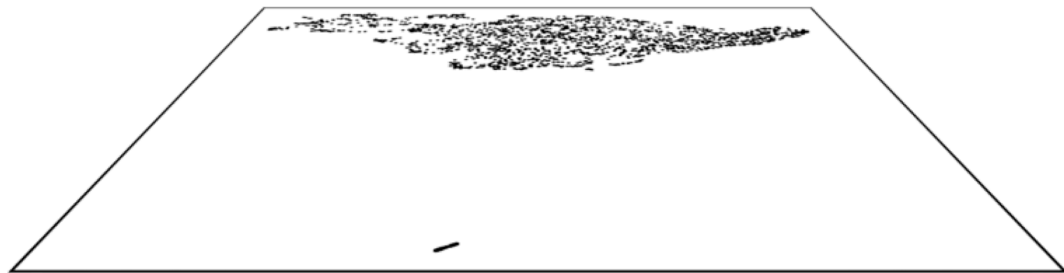
quartz_off_screen
2

```

> ht <- (W - Wt)/2
> y_top <- 250
> H2 <- 450
>

```

```
> coords_shrunk <- c(0, 0, ht, y_top, W, 0, W - ht, y_top, 0, H, 0, H2, W, 1)
>
> img_trap <- image_distort(img, distortion = "perspective", coordinates =
+   bestfit = TRUE)
>
>
> plot(img_trap)
```

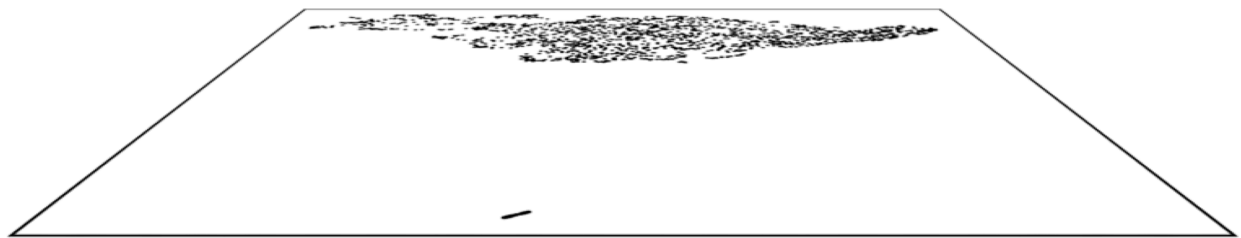
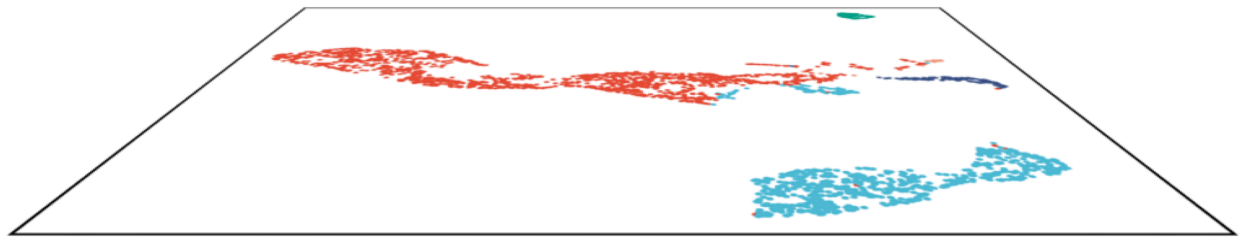


```

> g <- rasterGrob(img_trap, width = unit(1, "npc"), height = unit(1, "npc")
>
>
> p_query_trap <- ggdraw() + draw_grob(g, x = 0, y = 0.2, width = 1, height
+   theme(plot.margin = unit(c(0, 0, 0, 0), "pt"))

> p_trap <- p_onlyhighscore_trap + p_query_trap + plot_layout(ncol = 1)
>
> p_trap

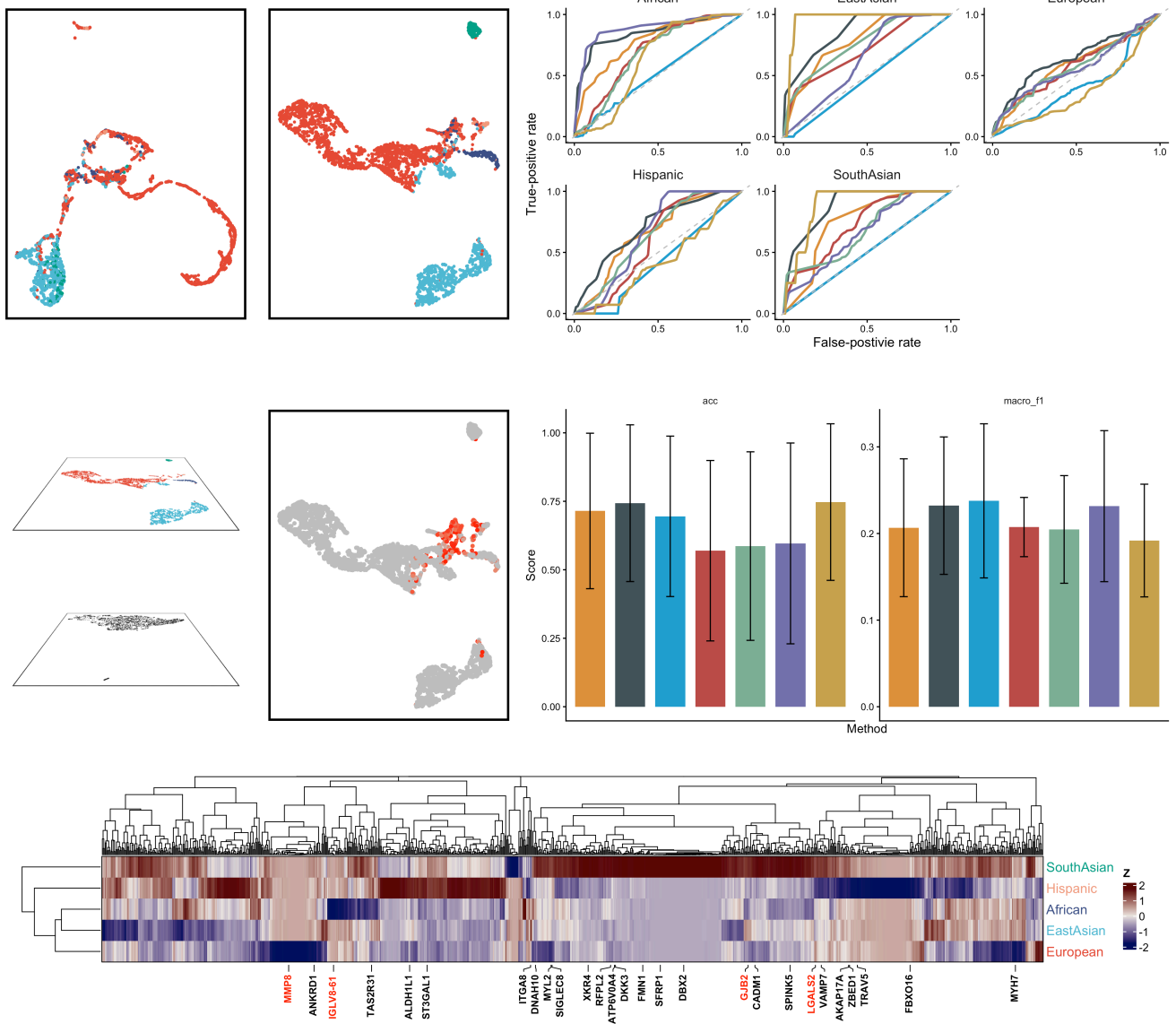
```



```
> ht_gg <- ggplotify::as.ggplot(function() {
+   draw(cht, newpage = FALSE, heatmap_legend_side = "right", annotation_
+ })
```

```
> layout <- "
+ AABBDddd
+ AABBDddd
+ FFCCGGGG
+ FFCCGGGG
+ EEEEEEEE
+ EEEEEEEE
+ EEEEEEEE
+ "
```

```
> p_before_batch_correct + p_post_batch_correct + p_scoring_umap + p_model_
+   ht_gg + p_trap + ov_performance + plot_layout(design = layout, height
+   3.5, 5, 2, 1, 5, 1, 2), widths = c(3, 1, 1, 3, 3, 3, 1))
```



```
> ggsave("../output/panel_figure.pdf", height = 14, width = 15, bg = "white")
```